

Enterprise Knowledge Based Automation

Complete Architecture & Operations Handbook

Tumhare apne project ka hands-on training manual — code, Terraform, scripts aur AWS deployment ke hisaab se, simple Hinglish mein.

Repository: `Analyst-Parveen/Enterprise-Knowledge-Based-Automation`

Snapshot: 2026-09-12 · branch `aws-deployment` · RDS migration ke baad

Table of Contents

1. Is Handbook Ko Kaise Use Karein	5
2. Project Overview — Ye Project Karta Kya Hai?	6
2.1 Problem kya hai?	6
2.2 Kaun use karta hai?	6
2.3 Important terms — simple bhasha mein	6
2.4 Real-world example	7
3. Complete Architecture	8
3.1 Poora system ek nazar mein	8
3.2 Kya live hai, kya nahi — honest table	9
4. Project Structure — Kaunsi File Kya Karti Hai?	11
4.1 Repository tree (important hisse)	11
4.2 Important files — ek-ek karke	12
5. AWS Services — Ye Service Kya Kar Rahi Hai?	15
5.1 Terraform — sab kuch banane wala	15
5.2 S3 — original documents	15
5.3 Cognito — user identity	16
5.4 ECS Fargate — backend chalane wali machine	16
5.5 ECR — Docker images ka godown	17
5.6 ALB — backend ka darwaaza	17
5.7 CloudFront — API ka HTTPS front door	18
5.8 Amplify — frontend hosting	18
5.9 CodeDeploy — blue/green release	18
5.10 Secrets Manager — secrets	19
5.11 CloudWatch — logs aur alarms	19
5.12 Bedrock — AI models	19
5.13 Amazon Transcribe	20
5.14 Lambda, SNS, EventBridge, Budgets — cost guard	20
5.15 IAM — kaun kya kar sakta hai	20
5.16 VPC & security groups	20
5.17 RDS — PostgreSQL database (managed)	20
5.18 Jo services is project mein NAHI hain	21
6. Authentication — Login Kaise Kaam Karta Hai	22
6.1 Flow — ek nazar mein	22
6.2 Basic terms	22
6.3 Frontend kya karta hai (actual code)	22
6.4 Backend kya karta hai (core/auth.py)	22
6.5 Tenant aur role kahan se aate hain	23
6.6 Discrepancy — kaunsa token paste karna hai?	23
6.7 Token expiry	23
7. Company / Tenant / User Creation	25
7.1 Seedhi baat pehle	25
7.2 Tenant ID kahan "rehta" hai?	25
7.3 Example: "ABC Insurance" ko onboard karna (currently supported tareeka)	25
7.4 Local mein users (no Cognito)	26
8. Admin vs User	27
8.1 Kaun kya kar sakta hai (code ke hisaab se)	27
8.2 Limits (dono roles par)	27
8.3 Scenario	28

9. Frontend — Browser Mein Kya Chal Raha Hai	29
9.1 Tech stack (frontend/package.json)	29
9.2 Do build modes	29
9.3 Pages (routes)	29
9.4 API client — src/lib/api.ts	30
9.5 Loading / empty / error states	30
9.6 Frontend → backend path	30
10. Backend — FastAPI Andar Se	31
10.1 Request ka safar (har request)	31
10.2 Endpoints (sab /api/v1 ke neeche)	31
10.3 Error format	31
11. Database — PostgreSQL	32
11.1 Technology	32
11.2 Tables (10)	32
11.3 ER diagram (simplified)	33
11.4 Indexes (tenant filter fast rakhne ke liye)	33
11.5 Migrations	33
12. Document Ingestion Flow	34
12.1 Diagram	34
12.2 Har stage	34
12.3 Status kaise badalta hai	35
12.4 Manually check karna	35
13. RAG Query Flow — Sabse Important Chapter	36
13.1 Example	36
13.2 Har step — Hinglish mein	36
13.3 Response envelope (frontend ko kya milta hai)	37
13.4 Hallucination kaise kam hota hai	37
13.5 Local mode vs AWS mode	37
13.6 Agentic workflows (LangGraph)	38
14. AI / LLM / Embeddings	39
14.1 Models	39
14.2 Pricing table (code mein, ai/registry.py, per 1M tokens)	39
14.3 Flow	39
15. Qdrant — Vector Database	41
16. Redis — Cache aur Rate Limit	42
17. Data Storage Map — Kya Kahan Rehta Hai	43
18. Local Development — Laptop Par Poora Project	44
18.1 Picture	44
18.2 Sabse aasaan raasta — ek command	44
18.3 Haath se, step by step (RUNBOOK Part A se verified)	44
18.4 Check, logs, docs	45
18.5 Stop / restart	45
19. AWS Production Flow	46
19.1 Frontend	46
19.2 Backend	46
19.3 Blue/green — kaise kaam karta hai	47
20. "Mujhe Code Change Karna Hai — Ab Kya Karu?"	48
20.1 Backend change	48
20.2 Frontend change	48
20.3 Decision tree	49
21. Branch Strategy — Asli Setup	50

22. Stop / Start / Destroy — Cheat Sheet	51
22.1 Stop vs Destroy	51
22.2 Commands	51
22.3 destroy.sh kya karta hai	51
22.4 Destroy ke baad redeploy	52
22.5 Rollback — galat version live ho gaya to kya karu?	52
23. Docker	53
24. Terraform — Beginner Guide	54
24.1 Basic commands	54
24.2 Chaar states	54
24.3 "Plan mein destroy/replace dikha — ab kya karu?"	54
24.4 State lock atka ho	55
25. CI/CD — GitHub Actions	56
25.1 ci.yml (automatic)	56
25.2 deploy.yml (manual, workflow_dispatch)	56
26. Testing	57
27. Monitoring & Logs — "Problem Aaya To Sabse Pehle Kahan Dekhu?"	58
28. Troubleshooting Playbook	59
28.1 "Kuch toot gaya" — decision tree	59
28.2 Problem → Check → Command → Wajah → Fix	59
29. Security	63
29.1 Controls — kya kahan	63
29.2 Honest gaps	63
29.3 GitHub mein kabhi commit nahi karna (.gitignore se)	64
30. Cost Guard — Kharcha Kaise Control Hota Hai	65
30.1 Pehle ek sach	65
30.2 Architecture	65
30.3 Thresholds	66
30.4 Shutdown kya karta hai / kya chhodta hai	66
30.5 DRY_RUN	66
30.6 Manually verify	67
30.7 Kharcha — numbers	67
31. Manual Data Inspection (read-only)	68
32. Real End-to-End Example — "ABC Insurance"	70
33. Complete Command Cheat Sheet	71
34. Interview Preparation	74
34.1 Two-minute version	74
34.2 Five-minute version — structure	74
34.3 Likely questions — short answers	74
35. NEVER DO THIS	76
36. Glossary	77
37. Final Quick Reference	79

1. Is Handbook Ko Kaise Use Karein

Parveen, ye handbook tumhare **apne repository** ko padh kar likhi gayi hai — koi generic AWS ya RAG tutorial nahi. Har command, har file path, har AWS resource ka naam code, Terraform aur scripts se verify kiya gaya hai.

Teen rules jo is handbook mein follow kiye gaye hain:

Rule	Matlab
Code hi source of truth hai	Agar docs aur code alag baat kehte hain, to dono likhe gaye hain aur bataya gaya hai ki code kya karta hai
"Implemented nahi hai" honestly likha hai	Jo cheez repo mein nahi hai, uske liye likha hai: Project mein currently ye implemented nahi hai.
Koi secret nahi	Passwords, tokens, keys, account ID — kuch nahi. Unki jagah <code><YOUR_...></code> placeholders hain

Kaise padhein:

- Pehli baar: Chapter 2 se 5 tak seedha padh lo — project samajh aa jayega.
- Kuch karna hai (deploy, change, stop): Chapter 18–24 aur 33 (cheat sheet).
- Kuch toot gaya: Chapter 28 (troubleshooting) aur Chapter 27 (logs).
- Interview se pehle: Chapter 34.

Box ke rang ka matlab: neela = samjhane wali baat · peela = dhyaan do · laal = ye kabhi mat karna · hara = verified fact / achhi practice.

Is snapshot ki live state (2026-09-12):

Cheez	State
AWS account plan	Paid (Free plan CodeDeploy block karta hai)
Region	<code>us-west-2</code>
Backend stack (envs/dev)	Deployed — ECS + ALB + CodeDeploy blue/green
Database	Amazon RDS PostgreSQL 16 (<code>ekba-dev-postgres</code> , <code>db.t4g.micro</code> , private) — Postgres sidecar ki jagah. Task replace hone par data bachta hai (2026-09-12 ko row counts se verify kiya)
Frontend (Amplify)	Live: <code>https://aws-deployment.d39pgpamq0p0n8.amplifyapp.com</code>
API front door (CloudFront)	Live: <code>https://d2jw2wchz5oekh.cloudfront.net</code>
Cost guard	Applied, <code>DRY_RUN = true</code> (report karta hai, kuch rokta nahi). Ab RDS ko stop bhi karta hai (delete kabhi nahi)
Bedrock quota	0 — isliye AWS par chat answer aur demo seeding abhi nahi chalte

Ye URLs secret nahi hain, lekin ye state badal sakti hai. `destroy.sh` ke baad backend ALB ka naam badal jata hai; CloudFront aur Amplify URL same rehte hain (Chapter 22).

2. Project Overview — Ye Project Karta Kya Hai?

2.1 Problem kya hai?

Ek company ke paas bahut saare documents hote hain — HR policy, travel policy, NDA template, sales deck, safety SOP, spreadsheets, diagrams, recordings. Employee ko ek chhota sa sawaal poochna ho ("hotel reimbursement limit kitni hai?") to use poora PDF dhoondhna padta hai.

Ye project ek **private company knowledge assistant** hai:

- Company apne documents upload karti hai.
- System unhe padh kar, chhote tukdon (chunks) mein tod kar, searchable bana deta hai.
- Employee natural language mein sawaal poochta hai.
- System **sirf usi company ke** documents mein se relevant hissa dhoondh kar, AI model se answer banwata hai — **source (citation) ke saath**.

2.2 Kaun use karta hai?

Role	Kaun	Code mein kahan
<code>user</code>	Normal employee	<code>app/core/context.py</code> — <code>Role = "user" \ "admin"</code>
<code>admin</code>	Us company ka admin	Same; admin endpoints <code>app/api/v1/admin.py</code>

Sirf yahi do roles hain. Koi "super-admin" ya "platform owner" role code mein **nahi hai**.

2.3 Important terms — simple bhasha mein

Multi-tenant — Ek hi application, kai companies use karein, aur har company ka data doosri se bilkul alag rahe. "Tenant" matlab ek company. Is project mein har company ka ek `tenant_id` hota hai (jaise `seed-tenant-northwind`).

RAG (Retrieval-Augmented Generation) — AI model se seedha answer poochne ke bajaye, pehle apne documents mein se relevant tukde **dhoondho (retrieve)**, phir wo tukde model ko do aur kaho "sirf inke basis par answer do". Isse hallucination (banayi hui baat) kam hoti hai.

Agentic RAG — Ek se zyada step wala workflow jahan system khud plan karta hai, retrieve karta hai, analyze karta hai, phir answer banata hai. Ye project mein **implemented hai**: LangGraph graph `plan → retrieve → analyze → synthesize → cite` (`app/services/agents/graph.py`), 5 workflows ke saath (Chapter 13.6).

Citations — Answer ke saath ye batana ki ye baat kis document ke kis hisse se aayi. Model answer mein `[S1]`, `[S2]` jaise markers lagata hai; backend check karta hai ki wo marker sach mein retrieve kiye gaye chunk ka hai (`app/services/rag/guardrails.py → validate_citations`).

Guardrails — Suraksha ki deewarein. Input ke liye (prompt injection check), aur output ke liye (system prompt leak, secrets, unsafe HTML). Is project mein ye pipeline ke **compulsory stages** hain, optional nahi.

Tenant isolation — Company A ka user kabhi bhi Company B ka ek bhi document, chunk, cache entry ya answer na dekh sake. Ye is project ka **sabse important rule** hai.

2.4 Real-world example

```
COMPANY (tenant: seed-tenant-northwind)
|
|— Department (hr, finance, legal, sales, marketing, operations, technical)
|
|— Admin / User (Cognito identity: custom:tenant_id + custom:role)
|
|— Documents (S3 mein file, PostgreSQL mein metadata)
|
|— Knowledge Base (Qdrant mein chunks + embeddings, har chunk par tenant_id)
    |
    ▼
    User sawaal poochta hai: "Domestic hotel reimbursement limit kya hai?"
    |
    ▼
    System SIRF northwind ke chunks mein search karta hai
    |
    ▼
    LLM (Bedrock Nova) answer banata hai
    |
    ▼
    Answer + citation: "150 USD [S1]" → Travel Policy 2026.pdf, page 4
```

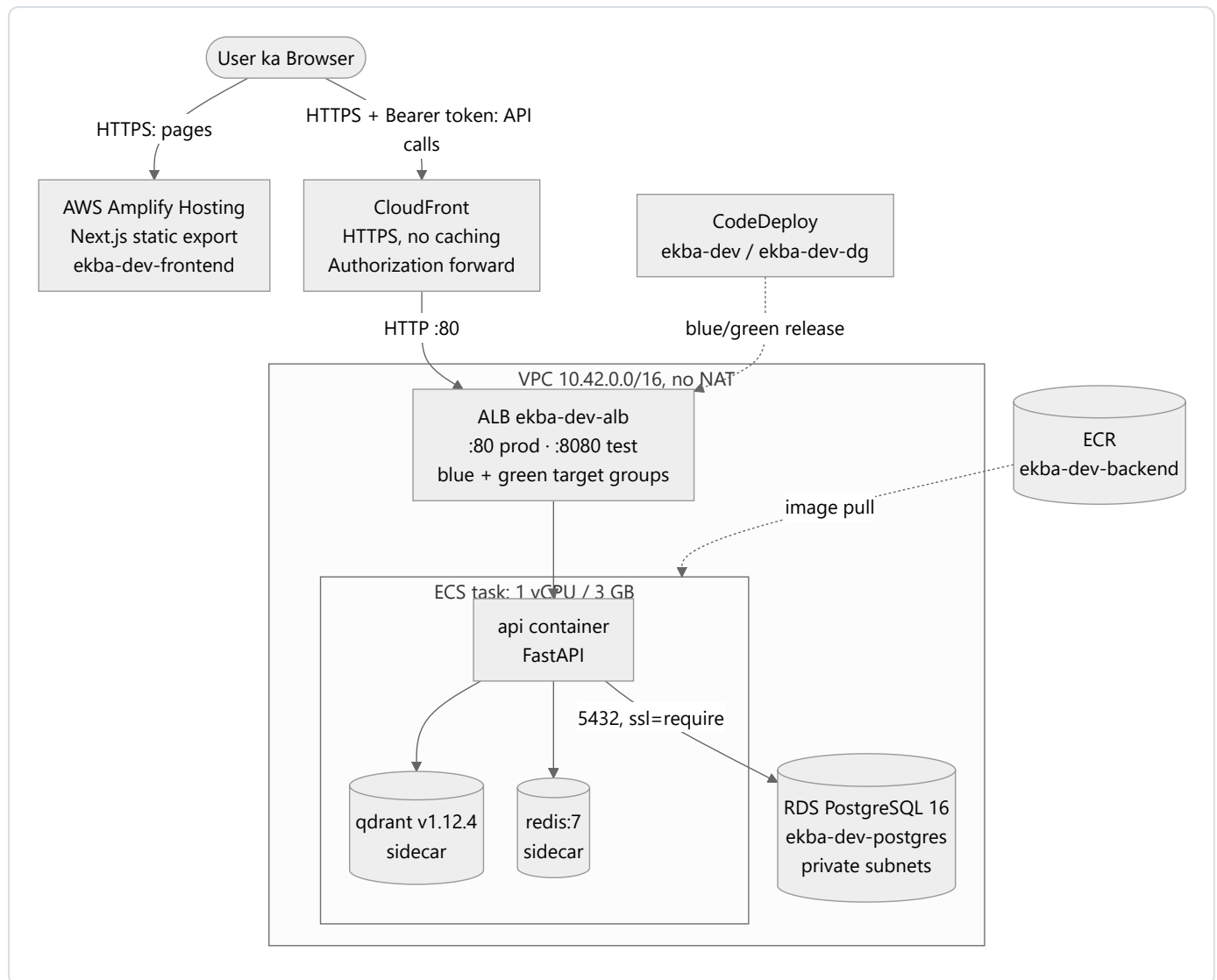
Verified: ye exact example seed data mein hai — `python -m seeds.seed "Travel Policy 2026.pdf"` banata hai aur RUNBOOK ke hisaab se answer "150 USD" aata hai (local mode mein).

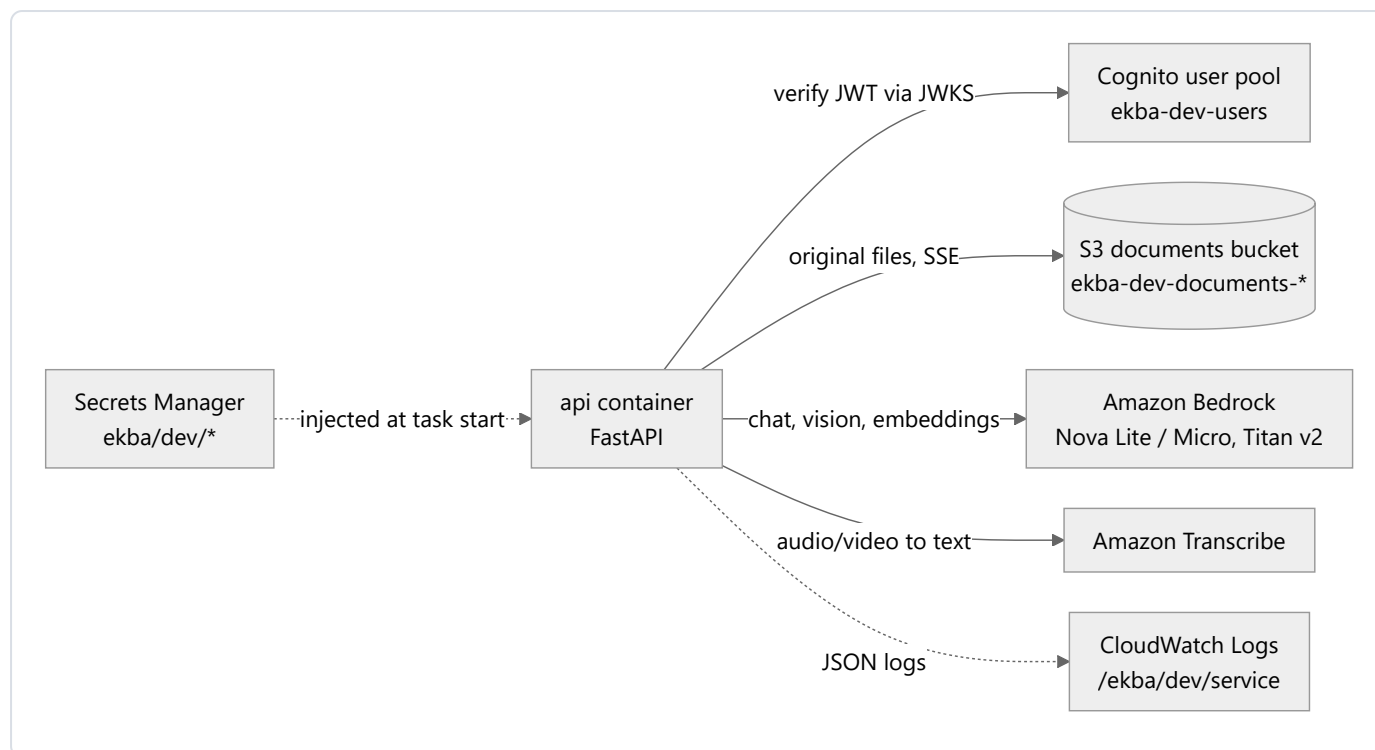
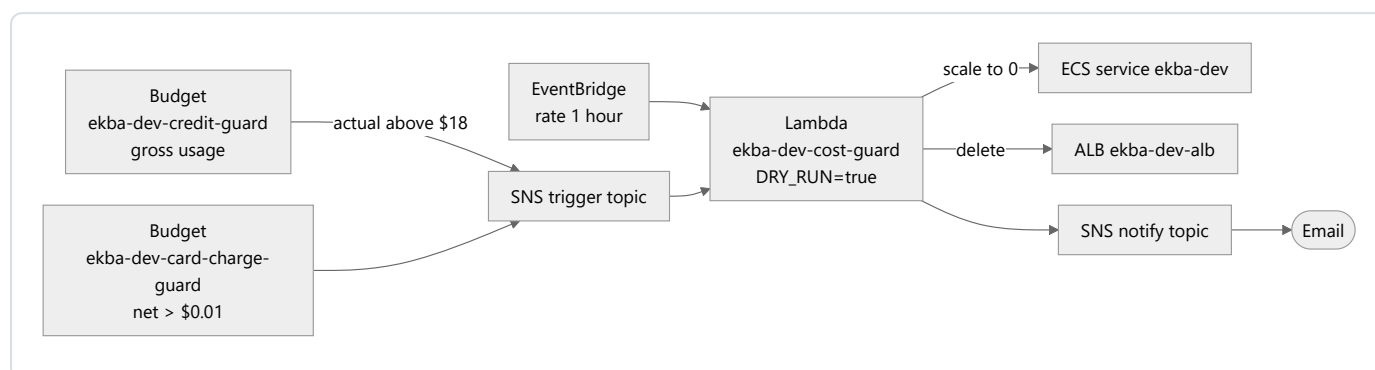
3. Complete Architecture

3.1 Poora system ek nazar mein

Neeche ke diagrams **sirf wahi components dikhate hain jo repository mein actually configure hain**. Kaunsa component live hai aur kaunsa sirf configured / blocked (jaise Bedrock quota 0), wo section 3.2 ki table mein hai.

(A) Request path — browser se containers tak:



(B) api container kin AWS services se baat karta hai:**Cost guard aur baaki control plane** (alag diagram — Chapter 30):**3.2 Kya live hai, kya nahi — honest table**

Component	Status	Detail
Amplify frontend	Live	Static export, <code>aws-deployment</code> branch se auto-build
CloudFront API	Live	HTTPS layer, ALB HTTP-only hai isliye
ALB + ECS Fargate	Live	Ek task, teen containers (api, qdrant, redis)
PostgreSQL	Live (Amazon RDS)	<code>ekba-dev-postgres</code> , <code>db.t4g.micro</code> , private subnets. Data task restart par bachta hai ; <code>destroy.sh</code> snapshot leta hai
Supabase	Implemented nahi hai	Project mein currently ye implemented nahi hai
Qdrant	Live (container)	Sidecar, koi managed vector DB nahi
Redis	Live (container)	Sidecar. ElastiCache nahi

Component	Status	Detail
S3	Live	Documents bucket (baseline state)
Cognito	Configured	Pool + client bane hain; AWS par user login abhi live-test nahi hua (Chapter 6.6)
Bedrock	Configured, blocked	Account quotas 0 — chat/embedding AWS par throttle hote hain
Transcribe	Configured	Audio/video ingestion ke liye; AWS par test nahi hua
CodeDeploy	Live	Blue/green deployments successful
CloudWatch Logs	Live	<code>/ekba/dev/service</code> (1 din retention)
CloudWatch Alarms	Live	<code>ekba-dev-5xx</code> , <code>ekba-dev-latency</code>
Cost guard (Budgets, SNS, Lambda, EventBridge)	Live, DRY_RUN	Report karta hai, act nahi karta
LangSmith	Configured only	Setting + secret hai, lekin tracing ka code implemented nahi hai
SQS	Implemented nahi hai	Project mein currently ye implemented nahi hai
EKS / NAT Gateway / ElastiCache / EFS	Kabhi nahi	Cost rule ke hisaab se Terraform mein allowed hi nahi
RDS Multi-AZ / bada instance / doosra database	Kabhi nahi	Sirf ek <code>db.t4g.micro</code> allowed hai (Chapter 11)

4. Project Structure — Kaunsi File Kya Karti Hai?

4.1 Repository tree (important hisse)

```
Enterprise-Knowledge-Based-Automation/
├─ README.md, RUNBOOK.md, PROJECT.md, CLAUDE.md  ← docs + rules summary
├─ amplify.yml                                  ← Amplify build spec (frontend)
├─ .env.example                                ← local config template (placeholders only)
├─ .github/workflows/
│   └─ ci.yml                                  ← tests/lint/scan on push to main/develop
│   └─ deploy.yml                              ← manual AWS deploy (abhi tak run nahi hua)
├─ .claude/rules/ .claude/skills/ ← project ke binding rules + workflows
├─ backend/
│   └─ app/
│       └─ main.py                            ← FastAPI app, middleware, routes
│       └─ core/                              ← config, auth, context, logging, middleware, ratelimit
│       └─ api/deps.py                        ← auth + rate-limit dependencies
│       └─ api/v1/                            ← health, documents, chat, agents, admin
│       └─ db/                                ← models.py, repositories.py, session.py
│       └─ services/
│           └─ rag/                            ← pipeline, cache, guardrails, prompts, rerank
│           └─ ingestion/                      ← pipeline, extractors, chunker
│           └─ ai/                             ← provider (Bedrock/local), registry, transcribe
│           └─ agents/                         ← LangGraph workflows
│           └─ security/                       ← injection scan, file safety
│           └─ observability/                  ← evaluation metrics
│           └─ vector.py                       ← Qdrant (tenant filter)
│           └─ storage.py                      ← S3 / MinIO
│       └─ schemas.py                         ← Pydantic request/response models
│       └─ workers/                           ← sirf __init__.py – koi separate worker nahi
├─ alembic/versions/0001_initial_schema.py    ← DB migration
├─ seeds/seed.py, documents.py, dev_token.py ← demo data + local token
├─ tests/unit|integration|security|evaluation
├─ frontend/
│   └─ next.config.mjs                        ← export (Amplify) vs standalone (Docker)
│   └─ src/app/                              ← pages (dashboard, chat, documents, admin/...)
│   └─ src/components/                       ← shell, ui, charts, admin
│   └─ src/lib/api.ts                         ← EK hi API client
│   └─ src/types/api.ts                      ← backend response types
│   └─ tests/e2e/journeys.spec.ts            ← Playwright (15 journeys)
├─ infra/
│   └─ docker/                              ← docker-compose.yml, backend/frontend Dockerfiles
│   └─ terraform/
│       └─ envs/baseline/                    ← protected: ECR, S3, Cognito, secrets, budget, OIDC
│       └─ envs/cost-guard/                  ← protected: kill switch
│       └─ envs/dev/                         ← ephemeral: VPC, ALB, ECS, CodeDeploy
│       └─ envs/frontend/                    ← persistent: Amplify + CloudFront
```

```

├── modules/network|service|cost-guard|frontend
├── scripts/                ← lifecycle scripts (deploy, destroy, verify...)
├── docs/reports/          ← har lifecycle run ki timestamped report

```

4.2 Important files — ek-ek karke

Har file ke liye: **Purpose** → **Kaun call karta hai** → **Ye kya call karti hai** → **Change ka asar** → **Kab edit karna** → **Kaise test karna**.

Backend core

File	Purpose	Called by	Calls	Edit kab?	Test kaise?
<code>backend/app/main.py</code>	FastAPI app banata hai, middleware order, 5 routers, startup par Qdrant collection ensure	uvicorn	<code>core/*</code> , <code>api/v1/*</code> , <code>vector.ensure_collection</code>	Naya router/middleware	<code>pytest tests/integration</code>
<code>backend/app/core/config.py</code>	Sabhi settings ek jagah (<code>Settings</code>), <code>.env</code> repo root se padhta hai	Har module	pydantic-settings	Naya config variable	<code>pytest tests/unit</code> + <code>.env.example</code> update
<code>backend/app/core/auth.py</code>	JWT verify (Cognito RS256 ya local dev HS256), tenant + role nikalna	<code>api/deps.py</code>	Cognito JWKS	Auth rules	<code>pytest tests/security/test_platform_security.py</code>
<code>backend/app/api/deps.py</code>	<code>CurrentUser</code> , <code>AdminUser</code> , rate-limited users — har route ki "gate"	Har route	<code>auth.verify_token</code> , <code>ratelimit.enforce</code>	Naya permission type	security tests
<code>backend/app/core/middleware.py</code>	Correlation ID, security headers, trusted hosts, body size limit	<code>main.py</code>	—	Header/limit badalna	integration tests
<code>backend/app/core/ratelimit.py</code>	Redis par per-user rate limit (20/10/5 per min)	<code>deps.py</code>	Redis	Limits	security tests

Data layer

File	Purpose	Edit kab?
<code>backend/app/db/models.py</code>	10 tables ka SQLAlchemy model	Naya column/table → saath mein Alembic migration zaroori
<code>backend/app/db/repositories.py</code>	Tenant-filtered data access (documents, jobs, usage, audit)	Naya query — hamesha <code>tenant_id</code> filter ke saath
<code>backend/alembic/versions/0001_initial_schema.py</code>	Poora schema banane wali migration	Kabhi edit mat karo agar kahin apply ho chuki; nayi migration banao

Services (business logic)

File	Kya karti hai
<code>services/rag/pipeline.py</code>	RAG ka dil — 14 stage wala <code>answer_question</code>
<code>services/rag/guardrails.py</code>	Citation validation, output guardrail, confidence
<code>services/rag/cache.py</code>	Redis semantic cache (tenant-namespaced)
<code>services/rag/rerank.py</code>	BM25-style lexical rerank + vector score fusion
<code>services/rag/prompts.py</code>	System prompt (versioned in code), context envelope
<code>services/security/injection.py</code>	Prompt injection scanner (10 pattern families)
<code>services/security/files.py</code>	Upload allow-list, magic bytes, size, safe S3 key
<code>services/ingestion/pipeline.py</code>	Extract → chunk → scan → embed → Qdrant
<code>services/ingestion/extractors.py</code>	PDF, DOCX, CSV, Excel, MD, TXT, image, audio/video
<code>services/ingestion/chunker.py</code>	Paragraph/sentence boundary chunking with overlap
<code>services/vector.py</code>	Qdrant — collection, upsert, mandatory tenant filter search
<code>services/storage.py</code>	S3/MinIO — <code><tenant_id>/<uuid>.<ext></code> keys, presigned URLs
<code>services/ai/provider.py</code>	Bedrock provider + <code>\$0</code> local provider
<code>services/ai/registry.py</code>	Model roles → model IDs, pricing, vision check
<code>services/ai/transcribe.py</code>	Amazon Transcribe job + polling
<code>services/agents/graph.py</code> , <code>nodes.py</code>	LangGraph agentic workflows

Frontend

File	Purpose	Edit kab?
<code>frontend/src/lib/api.ts</code>	Ekmatra API client — token, correlation ID, errors	Naya endpoint call
<code>frontend/src/components/shell.tsx</code>	Session (sign-in/out), sidebar nav, page header	Nav item, login screen
<code>frontend/src/components/ui/index.tsx</code>	Buttons, cards, badges, tables, stat tiles	Design system
<code>frontend/src/components/charts.tsx</code>	SegmentedBar, BarList, RingMeter (no library)	Chart badlav
<code>frontend/src/app/dashboard/page.tsx</code>	Dashboard (real API data)	Dashboard
<code>frontend/next.config.mjs</code>	<code>NEXT_OUTPUT_MODE=export</code> → static; warna standalone	Build mode
<code>amplify.yml</code>	Amplify build commands	Amplify build badalna

Infra & scripts

File	Purpose
<code>infra/terraform/envs/*/main.tf</code>	Chaar Terraform states (Chapter 24)
<code>infra/terraform/modules/service/main.tf</code>	ALB, ECS task (3 containers), CodeDeploy, IAM, alarms
<code>infra/terraform/modules/database/main.tf</code>	RDS PostgreSQL, DB subnet group (write-only password)

File	Purpose
<code>infra/terraform/modules/network/main.tf</code>	VPC, subnets, security groups (NO NAT)
<code>infra/terraform/modules/cost-guard/</code>	Budgets, SNS, Lambda (<code>lambda/handler.py</code>), EventBridge
<code>infra/terraform/modules/frontend/main.tf</code>	Amplify app/branch, CloudFront
<code>scripts/_common.sh</code>	Saare scripts ke shared guards (account check, cost guard, reports)
<code>scripts/deploy.sh</code>	Backend deploy pipeline
<code>scripts/deploy-frontend.sh</code>	Frontend deploy + wiring + verification
<code>scripts/destroy.sh</code>	Ekmatra <code>terraform destroy</code> path

`backend/app/workers/` **khali hai** (sirf `__init__.py`). Docs "async ingestion workers" bolte hain, lekin code mein ingestion **FastAPI BackgroundTasks** se use API process ke andar chalta hai (`api/v1/documents.py` → `_run_ingestion_task`). Koi alag worker/queue nahi hai.

5. AWS Services — Ye Service Kya Kar Rahi Hai?

Har service ke liye: **kyun chahiye** · **kya store/run karti hai** · **input/output** · **kis se judi hai** · **console mein kahan dekhein** · **CLI** · **down ho to kya** · **cost** · **interview line**.

Region hamesha `us-west-2`. Resource naam Terraform se hain: `ekba-<env>-<component>` (yahan `env = dev`). Account ID ki jagah `<ACCOUNT_ID>` likha hai.

5.1 Terraform — sab kuch banane wala

Kaam: AWS infrastructure ko code se create/manage karna. Console mein haath se ECS, ALB, S3 banane ke bajaye, Terraform wahi cheezein repeatable tareeke se banata hai. **Kahan:** `infra/terraform/` — chaar alag **states** (Chapter 24). **State kahan rehti hai:** S3 bucket `ekba-tfstate-<ACCOUNT_ID>` + lock ke liye DynamoDB table `ekba-tfstate-lock` (dono `scripts/bootstrap-state.sh` banata hai, Terraform ke bahar). **Interview line:** "Infra ko chaar states mein baanta — protected baseline, protected cost guard, ephemeral dev, persistent frontend — taaki `destroy` kabhi secrets ya ECR images tak pahunch hi na sake."

5.2 S3 — original documents

Kyun	Uploaded original files rakhne ke liye (sirf persistent data store)
Resource	<code>ekba-dev-documents-<random-suffix></code> (baseline state)
Security	Public access block , SSE <code>AES256</code> , versioning on , TLS-only bucket policy, <code>prevent_destroy</code>
Key format	<code><tenant_id>/<uuid>.<ext></code> — server banata hai, filename se kabhi nahi (<code>services/security/files.py</code> → <code>build_storage_key</code>)
Input	Upload API se file bytes
Output	Ingestion ke liye bytes; download ke liye 300 second ka presigned URL
Local mein	S3 ki jagah MinIO container (<code>S3_ENDPOINT_URL=http://localhost:9000</code>)

Admin/User upload → POST `/api/v1/documents` → `files.py` checks → S3 put (SSE) → Document row → ingestion

Console: **S3** → **Buckets** → `ekba-dev-documents-...` → folders tenant ID ke naam se.

```
aws s3 ls s3://<BUCKET>/ # tenant prefixes
aws s3 ls s3://<BUCKET>/seed-tenant-northwind/ # ek tenant ki files
terraform -chdir=infra/terraform/envs/baseline output -raw s3_bucket # bucket ka naam
```

Down ho to: upload aur download fail; baaki search chalta rahega (chunks Qdrant mein hain). Cost: pennies.

AWS par ek important baat (2026-09-12 se badla hai): file S3 mein bachti hai **aur** uska metadata ab **RDS PostgreSQL** mein bachta hai — task restart, naya deploy, sab ke baad. Lekin **chunks/vectors (Qdrant) abhi bhi ephemeral sidecar** mein hain, isliye task replace hone par vectors chale jaate hain: document list mein dikhega, par uska jawab RAG mein nahi aayega jab tak dobara upload na karo. Seed ke apne documents har startup par khud re-index ho jaate hain. (Re-index job project mein currently implemented nahi hai.)

5.3 Cognito — user identity

Kyun	Login identity + JWT token dena
Resource	User pool <code>ekba-dev-users</code> , app client <code>ekba-dev-web</code> (baseline state)
Username	Email (<code>username_attributes = ["email"]</code>)
Custom attributes	<code>custom:tenant_id</code> (1–64 chars), <code>custom:role</code> (1–16 chars) — saara authorization data yahi hai
Password policy	Min 12 chars, upper + lower + number + symbol
MFA	<code>OFF</code> ("demo scope")
App client	Public client (no secret), flows: <code>USER_PASSWORD_AUTH</code> , <code>USER_SRP_AUTH</code> , <code>REFRESH_TOKEN_AUTH</code>
Token validity	Access 1 hour, ID 1 hour, Refresh 7 days
Hosted UI domain	Configure nahi hai — Terraform mein <code>aws_cognito_user_pool_domain</code> nahi hai

Console: **Cognito** → **User pools** → **ekba-dev-users** → **Users**.

```
PPOOL=$(terraform -chdir=infra/terraform/envs/baseline output -raw cognito_user_pool_id)
aws cognito-idp list-users --user-pool-id "$PPOOL" --query 'Users[][Username,UserStatus]' --output table
```

Down ho to: naye login nahi ho payenge; backend JWKS se token verify nahi kar payega → 401. Cost: is scale par free tier.

5.4 ECS Fargate — backend chalane wali machine

Kyun	Backend container(s) chalane ke liye bina server manage kiye
Cluster / service	<code>ekba-dev</code> / <code>ekba-dev</code> , launch type <code>FARGATE</code> , deployment controller <code>CODE_DEPLOY</code>
Task size	1 vCPU (<code>1024</code>) / 3 GB (<code>3072</code>), x86_64
Networking	Public subnet + public IP (NAT Gateway nahi, taaki ECR/Bedrock tak pahunch sake); security group sirf ALB se traffic leta hai
Containers (ek hi task mein)	<code>api</code> (FastAPI), <code>qdrant/qdrant:v1.12.4</code> , <code>redis:7-alpine</code> — PostgreSQL ab RDS hai (5.17)
<code>api</code> startup command	<code>alembic upgrade head && (python -m seeds.seed \\ \\ echo 'seed failed ...') && exec uvicorn</code> ...

Health checks	api: <code>curl /api/v1/health</code> ; redis: <code>redis-cli ping</code> ; qdrant: koi health check nahi (isliye status UNKNOWN dikhta hai)
Container Insights	<code>disabled</code> (cost bachane ke liye)

```
aws ecs describe-services --cluster ekba-dev --services ekba-dev \
  --query 'services[0].[desiredCount,runningCount,taskDefinition]' --output text
aws ecs list-tasks --cluster ekba-dev --desired-status RUNNING
```

Down ho to: poora API down (ALB 503/504). Cost: ~\$0.054/hour (Fargate 1 vCPU/3 GB list price).

5.5 ECR — Docker images ka godown

Repos	<code>ekba-dev-backend</code> (use hota hai), <code>ekba-dev-frontend</code> (bana hai, koi push nahi karta — frontend Amplify se deploy hota hai)
Tags	IMMUTABLE — tag = git commit SHA (7 chars), <code>latest</code> kabhi nahi
Scan	<code>scan_on_push = true</code> ; <code>deploy.sh</code> critical findings par ruk jata hai
Lifecycle	Untagged 1 din mein expire; last 10 images rakho

```
aws ecr describe-images --repository-name ekba-dev-backend \
  --query 'sort_by(imageDetails,&imagePushedAt)[-5:].[imageTags[0],imagePushedAt]' --output table
```

ECR images **manually delete mat karo** — CodeDeploy rollback aur redeploy unhi par depend karte hain. ECR repo `prevent_destroy` aur protected hai.

5.6 ALB — backend ka darwaaza

Resource	<code>ekba-dev-alb</code> (internet-facing, 2 AZ), idle timeout 120 s
Listeners	<code>:80</code> production, <code>:8080</code> test (CodeDeploy green ko yahan check karta hai)
Target groups	<code>ekba-dev-blue</code> , <code>ekba-dev-green</code> — har deployment mein roles badalte hain; health check <code>GET /api/v1/health</code> → 200
Security group	Sirf <code>allowed_cidrs</code> (tumhara ek <code>/32</code> IP) port 80/443 par + CloudFront origin-facing prefix list port 80 par. Port 8080 kisi ke liye open nahi
HTTPS	Nahi — ALB par certificate/HTTPS listener nahi hai; HTTPS CloudFront deta hai

```
ALB=$(aws elbv2 describe-load-balancers --names ekba-dev-alb --query 'LoadBalancers[0].LoadBalancerArn' --
output text)
aws elbv2 describe-listeners --load-balancer-arn "$ALB" --query 'Listeners[0].
[Port,DefaultActions[0].TargetGroupArn]' --output text
```

Down ho to: CloudFront 502/504 dega. Cost: ~\$0.023/hour + LCU + 2 public IPv4.

5.7 CloudFront — API ka HTTPS front door

Kyun: Amplify site HTTPS par hai, ALB HTTP-only hai. Browser HTTPS page se HTTP API call block karta hai (mixed content), aur Amplify ka reverse proxy sirf HTTPS targets support karta hai. Isliye CloudFront (default *.cloudfront.net certificate) API ko HTTPS URL deta hai — **domain naam ki zaroorat nahi**.

Setting	Value (modules/frontend/main.tf)
Cache policy	ekba-dev-api-no-cache — TTL 0 (max 1 s), Authorization cache key mein (GET par forward karne ka yahi tareeka hai; kisi aur user ko response kabhi nahi milega)
Origin request policy	Managed-AllViewerExceptHostHeader — Host ALB ka naam rehta hai, jo TRUSTED_HOSTS (*.elb.amazonaws.com) pass karta hai
Viewer	https-only , PriceClass_100
Origin	ALB DNS, http-only , read timeout 60 s

URL stable rehta hai; sirf origin (ALB naam) deploy-frontend.sh update karta hai. Cost: demo traffic par ~\$0.

5.8 Amplify — frontend hosting

App	ekba-dev-frontend (platform WEB = static hosting), repo Analyst-Parveen/Enterprise-Knowledge-Based-Automation
Branch	aws-deployment , stage PRODUCTION , framework Next.js - SSG , auto-build on push
Build spec	Repo ka amplify.yml (app root frontend , output out/)
Env vars (Terraform)	AMPLIFY_MONOREPO_APP_ROOT=frontend , NEXT_PUBLIC_API_URL=https://<cloudfront> , NEXT_TELEMETRY_DISABLED=1
Custom headers	HSTS, nosniff, X-Frame-Options DENY, Referrer-Policy, CSP (monorepo format applications[].appRoot)
Rule	/<*> → /404.html (404)
GitHub connection	AWS Amplify GitHub App (browser se ek baar authorize) — koi GitHub token Terraform state mein nahi

Console: **Amplify** → **ekba-dev-frontend** → **aws-deployment** → **Deployments** (har job ke build logs).

```
aws amplify list-jobs --app-id <APP_ID> --branch-name aws-deployment \
--query 'jobSummaries[][.jobId,status,commitId]' --output text
```

Cost: ~\$0.04 per build (~4 build-minutes), idle ~\$0.

5.9 CodeDeploy — blue/green release

App / group	ekba-dev / ekba-dev-dg , config CodeDeployDefault.ECSAllAtOnce
Style	BLUE_GREEN , WITH_TRAFFIC_CONTROL
Rollback window	Purana task set 5 minute (rollback_window_minutes) zinda rehta hai

Auto rollback	<code>DEPLOYMENT_FAILURE</code> aur alarm <code>ekba-dev-5xx</code> par (configure hai, abhi tak exercise nahi hua)
Requirement	Account Paid plan par hona chahiye (Free plan par <code>SubscriptionRequiredException</code>)

```
aws deploy list-deployments --application-name ekba-dev --deployment-group-name ekba-dev-dg --max-items 3
aws deploy get-deployment --deployment-id <DEPLOYMENT_ID> --query 'deploymentInfo.status'
```

5.10 Secrets Manager — secrets

Chaar containers (baseline, `recovery_window_in_days = 30`, `prevent_destroy`): `ekba/dev/backend/database-url` (JSON: `url`, `password`), `ekba/dev/backend/qdrant-api-key`, `ekba/dev/backend/dev-auth-secret`, `ekba/dev/ai/langsmith-api-key`. Values `scripts/set-secrets.sh` bharta hai — **Terraform state mein kabhi nahi jaate**. ECS execution role task start par inject karta hai. Cost: \$0.40/secret/month ≈ \$1.60/month (baseline ka sabse bada idle kharcha).

```
aws secretsmanager list-secrets --query "SecretList[?starts_with(Name,'ekba/dev/')].Name" # sirf naam —
value kabhi print mat karo
```

5.11 CloudWatch — logs aur alarms

Log group	Retention	Kya hai
<code>/ekba/dev/service</code>	1 din	ECS containers (<code>api</code> , <code>qdrant</code> , <code>redis</code> stream prefixes)
<code>/ekba/dev/audit</code>	30 din	Audit logs (baseline, protected)
<code>/aws/lambda/ekba-dev-cost-guard</code>	14 din	Kill-switch Lambda

Alarms: `ekba-dev-5xx` (5 se zyada target 5xx/min, 2 minute), `ekba-dev-latency` (p95 > 10 s). Billing alarm `estimated_charges` sirf `us-east-1` mein banta — is deployment mein **exist nahi karta**; spend budgets cover karte hain.

```
MSYS_NO_PATHCONV=1 aws logs tail /ekba/dev/service --since 15m --follow
```

5.12 Bedrock — AI models

Chat primary `us.amazon.nova-lite-v1:0`, fallback `us.amazon.nova-micro-v1:0`, vision `us.amazon.nova-lite-v1:0`, embeddings `amazon.titan-embed-text-v2:0` (1024 dims). Details Chapter 14.

Current limitation: is account ke Bedrock quotas **0** hain (Titan Embeddings V2 requests/tokens per minute, Nova Lite cross-region requests, Nova Lite tokens/day). AWS par har Bedrock call `ThrottlingException` deta hai — isliye seed fail hota hai aur chat answer nahi deta. Fix: AWS Support case (Service limit increase → Amazon Bedrock, us-west-2).

5.13 Amazon Transcribe

Audio (mp3, wav, m4a, flac, ogg) aur video (mp4, mov, webm) ka transcript banata hai (services/ai/transcribe.py, max wait 15 min). Local mode mein bhi ye AWS call karta hai, isliye seed ka .mp4 locally failed dikhta hai.

5.14 Lambda, SNS, EventBridge, Budgets — cost guard

Poori detail Chapter 30. Short mein: 2 budgets + hourly EventBridge rule → SNS/Lambda ekba-dev-cost-guard → ECS scale to 0 + ALB delete. **Abhi** DRY_RUN=true .

5.15 IAM — kaun kya kar sakta hai

Role	Kaam
ekba-dev-execution	ECS task start: ECR pull, logs, sirf project ke secrets padhna
ekba-dev-task	App runtime: documents bucket objects, Bedrock invoke, Transcribe, Cognito read
ekba-dev-codedeploy	AWSCodeDeployRoleForECS
ekba-dev-github-deploy	GitHub Actions OIDC deploy role (scoped to this repo)
ekba-dev-cost-guard	Sirf ecs:UpdateService + DeleteLoadBalancer, woh bhi ephemeral tag wale ekba-dev resources par
AmplifySSRLoggingRole-...	Amplify console ne connect ke waqt banaya; static site ke liye use nahi hota; Terraform ignore karta hai

5.16 VPC & security groups

VPC 10.42.0.0/16 — **2 public subnets** (ALB + Fargate task, public IP ke saath) aur **2 private subnets** (10.42.10.0/24, 10.42.11.0/24) jinme sirf RDS hai aur koi internet route nahi. Internet gateway hai, **NAT Gateway nahi**. SGs: ekba-dev-alb (ingress: tumhara IP + CloudFront), ekba-dev-tasks (ingress: sirf ALB SG se port 8000/3000), ekba-dev-db (ingress: sirf tasks SG se port 5432, egress kuch nahi).

5.17 RDS — PostgreSQL database (managed)

- Kya hai:** AWS ka managed relational database. Pehle Postgres ek sidecar container tha; **2026-09-11 se ye RDS hai**.
- Is project mein kya karta hai:** saara relational data rakhta hai — tenants, users, documents, ingestion jobs, conversations, messages, request_usage, feedback, audit events.
- Naam:** ekba-dev-postgres · class db.t4g.micro · PostgreSQL 16 · 20 GB gp3 · single-AZ.
- Kahan rehta hai:** VPC ke **private subnets** mein (koi internet route nahi). publicly_accessible = false .
- Kaun connect kar sakta hai:** sirf backend task. SG ekba-dev-db port 5432 sirf ekba-dev-tasks SG se allow karta hai — koi CIDR nahi, koi IP nahi. Tumhara laptop seedha connect **nahi** kar sakta (yahi design hai).
- Password kahan se aata hai:** Secrets Manager ekba/dev/backend/database-url ki password key. Terraform use **ephemeral** resource se padh kar **write-only** password_wo mein deta hai → password na state mein jaata hai, na plan mein. Container ko wahi DB_PASSWORD milta hai aur wo khud DATABASE_URL banata hai (ssl=require).
- Migrations kahan chalti hain:** task ke startup par (alembic upgrade head). Schema already sahi ho to ye no-op hai.

8. **Data kab tak bachta hai:** task replace, naya deploy, task crash — sab mein **bachta hai**. `destroy.sh` par snapshot ban jaata hai aur agla `deploy.sh` usi se instance banata hai.
9. **Kharcha:** ~\$0.016/hour instance + ~\$0.003/hour storage = ~\$0.019/hour. 24/7 chale to ~\$14/month — isiliye ye ephemeral stack ka hissa hai, aur cost guard ise **stop** karta hai (delete kabhi nahi).
10. **Terraform:** `infra/terraform/modules/database`, `envs/dev` se use hota hai. Encrypted at rest, backups 1 din.
11. **Dekhne ka command:**

```
aws rds describe-db-instances --db-instance-identifier ekba-dev-postgres \
  --query 'DBInstances[0].[DBInstanceStatus,PubliclyAccessible,StorageEncrypted]' --output text
aws rds describe-db-snapshots --db-instance-identifier ekba-dev-postgres \
  --snapshot-type manual --query 'DBSnapshots[].DBSnapshotIdentifier' --output text
```

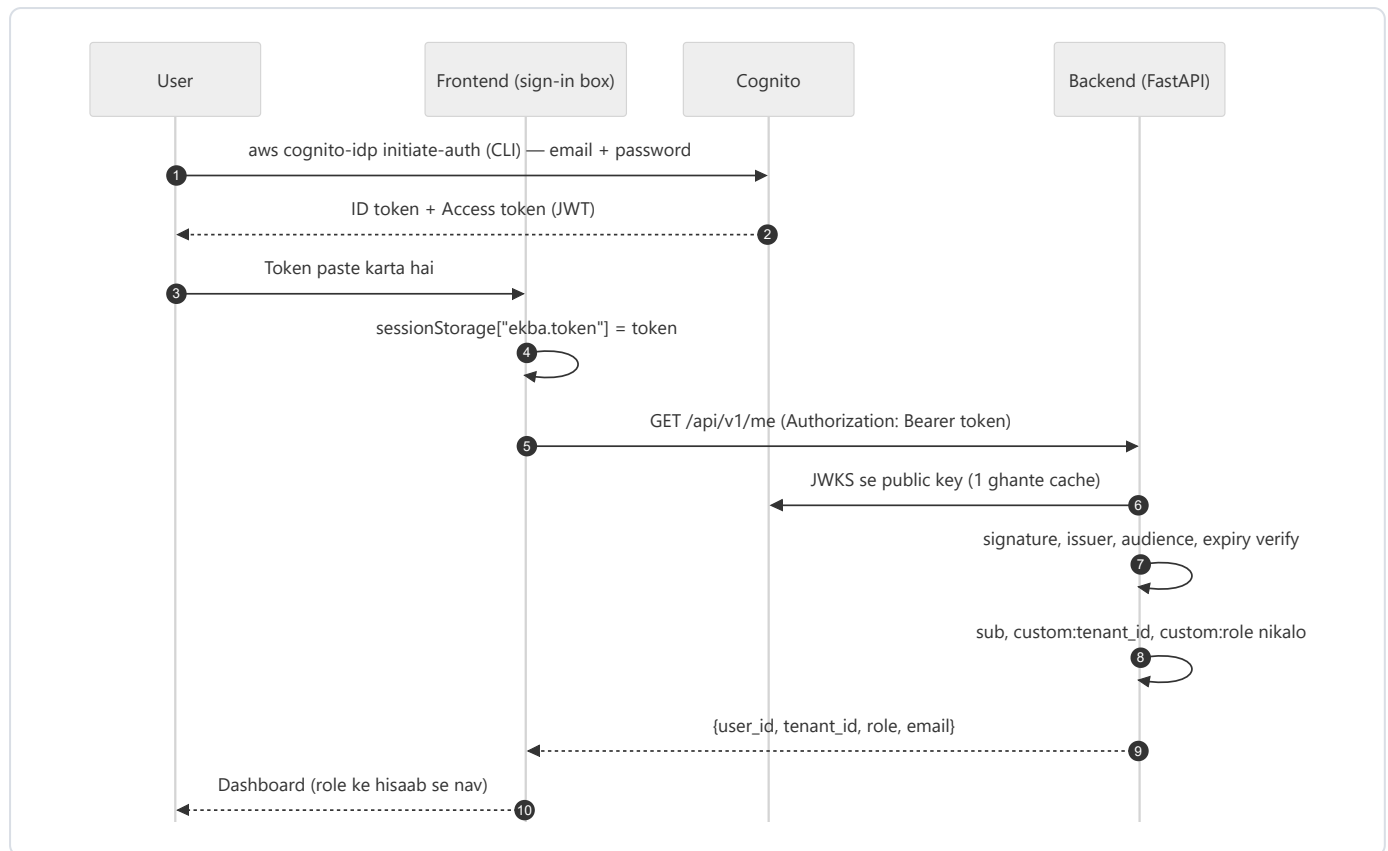
Dhyan do: Qdrant aur Redis **abhi bhi ephemeral sidecars** hain. Iska matlab: tumhare upload kiye document ki row (Postgres) bach jaati hai, par uske vectors (Qdrant) task replace hone par chale jaate hain — jawab aana band ho jayega jab tak dobara upload na karo. Seed ke documents har startup par khud re-index hote hain.

5.18 Jo services is project mein NAHI hain

Service	Status
Supabase	Project mein currently ye implemented nahi hai
Aurora / RDS Multi-AZ / RDS Proxy	Project mein currently ye implemented nahi hai — ek single-AZ <code>db.t4g.micro</code> hai
ElastiCache	Implemented nahi hai — Redis container
EKS, NAT Gateway, EFS	Rules ke hisaab se kabhi nahi
SQS	Project mein currently ye implemented nahi hai
Route 53 / custom domain / ACM certificate	Project mein currently ye implemented nahi hai
WAF	Project mein currently ye implemented nahi hai

6. Authentication — Login Kaise Kaam Karta Hai

6.1 Flow — ek nazar mein



6.2 Basic terms

Cognito — AWS ki user directory. Users, passwords aur tokens sambhalti hai. **JWT** — ek signed string (`header.payload.signature`). Payload mein claims hote hain (`sub`, `email`, `custom:tenant_id` ...). Signature ki wajah se koi ise badal nahi sakta. **ID token vs Access token** — Cognito dono deta hai. ID token mein user ke attributes (custom attributes bhi) aur `aud` (client ID) hota hai. Access token API permission ke liye hota hai aur usme custom attributes aam taur par nahi hote.

6.3 Frontend kya karta hai (actual code)

- Sign-in screen (`components/shell.tsx` → `SignIn`) mein ek **"Access token" box** hai — **koi username/password form nahi**. User token paste karta hai.
- Token `sessionStorage` key `ekba.token` mein jata hai (`lib/api.ts`) — tab band karo to token gaya.
- Har request par header: `Authorization: Bearer <token>` + `X-Correlation-ID`.
- API se **401** aaya to token delete (`clearToken`) → wapas sign-in screen.
- **Token refresh ka logic nahi hai** — token expire hone par naya token paste karna padega.

6.4 Backend kya karta hai (`core/auth.py`)

1. `api/deps.py` → `current_context` Bearer token nikalta hai; nahi hai to **401 "Authentication required."**

2. **Local dev mode** (`ENVIRONMENT=dev` aur `DEV_AUTH_ENABLED=true`): token HS256 se `DEV_AUTH_SECRET` ke saath verify (`seeds/dev_token.py` banata hai). Dono conditions na hon to ye path band.
3. **AWS mode**: Cognito JWKS (`https://cognito-idp.us-west-2.amazonaws.com/<POOL_ID>/well-known/jwks.json`) se key lekar **RS256** verify: signature, `exp`, `iss`, aur `aud = COGNITO_CLIENT_ID`.
4. Claims → `RequestContext`: - `user_id = sub` - `tenant_id = custom:tenant_id` (ya `tenant_id`) — **nahi mila to 401** ("Token is missing a tenant assignment.") - `role = custom:role` (default `user`); `user / admin` ke alawa kuch bhi → 401
5. Ye context poori request mein use hota hai — koi route khud token parse nahi karta.

Errors ka matlab:

Situation	Response	Security event
Token nahi bheja	401 <code>Authentication required.</code>	—
Bekaar/galat token	401 <code>Invalid token.</code>	<code>auth.invalid_token</code>
Expired	401 <code>Token has expired.</code>	<code>auth.token_expired</code>
Galat issuer/audience	401 <code>Invalid token.</code>	<code>auth.bad_issuer</code> / <code>auth.bad_audience</code>
Tenant claim missing	401	<code>auth.missing_tenant</code>
Admin route par non-admin	403	<code>authz.admin_required</code>

6.5 Tenant aur role kahan se aate hain

Sirf verified token se. Request body, query ya header mein `tenant_id` bhejne se kuch nahi hota — `ChatRequest` mein `tenant_id` field hai hi nahi (`schemas.py`: "deliberately no tenant_id field").

6.6 Discrepancy — kaunsa token paste karna hai?

RUNBOOK (Part B, Step 6) Cognito se `AuthenticationResult.AccessToken` lene ko kehta hai. **Code** (`core/auth.py`) `custom:tenant_id` claim maangta hai aur `aud` ko client ID se verify karta hai. Cognito ke standard tokens mein **ye dono ID token mein hote hain**, access token mein nahi.

Isliye code ke hisaab se **ID token** chahiye:

```
aws cognito-idp initiate-auth --client-id "$CLIENT" --auth-flow USER_PASSWORD_AUTH \
  --auth-parameters USERNAME=<EMAIL>,PASSWORD='<PASSWORD>' \
  --query 'AuthenticationResult.IdToken' --output text
```

Ye AWS par abhi tak live test nahi hua (Cognito users banaye nahi gaye). Pehli baar karte waqt agar `AccessToken` se "Token is missing a tenant assignment" aaye, to wajah yahi hai. Frontend ka sign-in text bhi "Cognito hosted UI" bolta hai — lekin hosted UI domain configure nahi hai.

6.7 Token expiry

Mode	Validity
Local <code>dev_token</code>	Default 12 ghante (<code>--hours</code>)

Mode	Validity
Cognito ID/Access token	1 ghanta (app client config)
Refresh token	7 din — lekin frontend refresh use nahi karta

7. Company / Tenant / User Creation

7.1 Seedhi baat pehle

Company registration ka koi UI ya API nahi hai. Project mein currently ye implemented nahi hai. Admin ka "Users" page sirf samjhata hai ki users Cognito mein bante hain. "Tenants" page sirf apne tenant ke metrics dikhata hai.

7.2 Tenant ID kahan "rehta" hai?

- Tenant ek **string** hai (jaise `seed-tenant-northwind`) jo user ke Cognito attribute `custom:tenant_id` mein likhi hoti hai.
- Database mein `tenants` aur `users` tables **hain** (`models.py`), aur `seeds/seed.py` unme demo rows daalta hai.
- Lekin **authentication in tables ko padhta hi nahi** — `repositories.get_user_by_sub` define hai par kahin call nahi hota. Tenant sirf token claim hai.
- Matlab: naya tenant "banana" = naye users ke `custom:tenant_id` mein ek naya, unique string likhna. Data us tenant ID ke saath apne-aap alag rehta hai (har table, har Qdrant point, har S3 key, har cache key mein tenant ID hota hai).

7.3 Example: "ABC Insurance" ko onboard karna (currently supported tareeka)

Step 1 — Tenant ID decide karo. Unique aur stable, jaise `tenant-abc-insurance`. (Koi DB insert zaroori nahi.)

Step 2 — Admin banao (Cognito, CLI):

```
POOL=$(terraform -chdir=infra/terraform/envs/baseline output -raw cognito_user_pool_id)

aws cognito-idp admin-create-user --user-pool-id "$POOL" \
  --username admin@abc-insurance.example \
  --user-attributes Name=email,Value=admin@abc-insurance.example Name=email_verified,Value=true \
    Name=custom:tenant_id,Value=tenant-abc-insurance Name=custom:role,Value=admin

aws cognito-idp admin-set-user-password --user-pool-id "$POOL" \
  --username admin@abc-insurance.example --password '<STRONG_PASSWORD>' --permanent
```

Step 3 — Normal user banao: same commands, `custom:role` = `user`.

Step 4 — Credentials dena: koi email/invite flow nahi hai — password tumhe khud securely dena padega (out-of-band).

Step 5 — Login: user `initiate-auth` se token leta hai (Chapter 6.6, **IdToken**) aur frontend mein paste karta hai.

Step 6 — Isolation: ab ABC ka har upload `tenant-abc-insurance/<uuid>` S3 key, us tenant ID ke DB rows, us tenant ID ke Qdrant points par jayega. Northwind ka user ise kabhi nahi dekh sakta.

Role badalna (promote to admin):

```
aws cognito-idp admin-update-user-attributes --user-pool-id "$POOL" \  
--username user@abc-insurance.example --user-attributes Name=custom:role,Value=admin
```

Naya role **agle login** (naye token) se lagega.

`custom:tenant_id` **galat likhna = galat company ka data access.** Tenant ID ek baar set ho gaya to use mat badlo. Aur ek identity ek hi tenant mein honi chahiye.

7.4 Local mein users (no Cognito)

`seeds/seed.py` ye banata hai:

user_id	Tenant	Role	Email
seed-user-a	seed-tenant-northwind	user	priya@northwind.example
seed-admin-a	seed-tenant-northwind	admin	—
seed-user-b	seed-tenant-contoso	user	—
seed-admin-b	seed-tenant-contoso	admin	—

Token:

```
cd backend  
python -m seeds.dev_token # northwind user  
python -m seeds.dev_token --role admin --user seed-admin-a # northwind admin  
python -m seeds.dev_token --tenant seed-tenant-contoso --user seed-admin-b --role admin
```

Documents sirf northwind mein seed hote hain — contoso admin se login karke tenant isolation demo dikhta hai (documents khali, chat "not found").

8. Admin vs User

8.1 Kaun kya kar sakta hai (code ke hisaab se)

Capability	USER	ADMIN	Code
Apne tenant ke documents dekhna (list/detail/status)	✓	✓	<code>documents.py</code> (rate-limited)
Document upload	✓	✓	<code>POST /documents</code> — <code>UploadUser</code> (admin-only nahi)
Document download (presigned URL, 5 min)	✓	✓	<code>GET /documents/{id}/download</code>
Document delete	✓ sirf apna (owner)	✓ tenant ka koi bhi	<code>repositories.soft_delete_document</code>
Knowledge chat (RAG)	✓	✓	<code>POST /chat</code>
Agentic workflows	✓	✓	<code>/agents/*</code> (10/min)
Feedback	✓	✓	<code>POST /feedback</code>
AI metrics (<code>/admin/metrics</code>)	✗ 403	✓	<code>admin.py</code>
Audit log (<code>/admin/audit</code>)	✗	✓	<code>admin.py</code>
Security events (<code>/admin/security</code>)	✗	✓	<code>admin.py</code>
Frontend admin pages (Users, Tenants, Documents, AI Metrics, Security, Audit, Deployments)	✗ "Administrator access required"	✓	<code>components/admin.tsx</code> → <code>AdminOnly</code>
Users create/manage	✗	✗ — app mein nahi; Cognito CLI/console	—
Tenant create/manage	✗	✗ — app mein nahi	—
Doosre tenant ka data	✗	✗ — admin bhi nahi	<code>tenant-isolation.md</code>

Admin ≠ super-user. Admin ko sirf apne tenant ka operational data milta hai. Frontend button chhupana authorization nahi hai — API khud check karta hai (`AdminUser` dependency).

8.2 Limits (dono roles par)

Limit	Value	Kahan
API requests	20 / minute / user	Redis <code>ekba:r1:api:<tenant>:<user></code>
Agent (server) requests	10 / minute	<code>ekba:r1:server:...</code>
Uploads	5 / minute (+ API bucket)	<code>ekba:r1:upload:...</code>
Upload size	50 MB	<code>S3_UPLOAD_MAX_BYTES</code>

Limit	Value	Kahan
Daily AI spend	\$0.50 / user / din	<code>DAILY_COST_CEILING_USD</code> (<code>request_usage</code> table se)

8.3 Scenario

```
Admin (northwind) login → Documents page → "Leave and Absence Policy.pdf" upload (dept: hr)
  → 201: document status "pending", job "queued"
  → background: extracting → chunking → embedding → ready
User (northwind) login → Knowledge Chat → "How much annual leave carries over?"
  → answer "5 days [S1]" + citation → Leave and Absence Policy.pdf
```

9. Frontend — Browser Mein Kya Chal Raha Hai

9.1 Tech stack (frontend/package.json)

Next.js **15.1.3** (App Router) · React 19 · TypeScript (strict) · Tailwind CSS 3.4 · lucide-react icons · koi chart library nahi (apne SVG/CSS charts).

9.2 Do build modes

Mode	Kab	Setting	Output
Static export	AWS Amplify	NEXT_OUTPUT_MODE=export (amplify.yml set karta hai)	frontend/out/ — har page ki HTML file (chat.html , ...)
Standalone server	Docker image (infra/docker/frontend.Dockerfile)	default	.next/standalone/server.js

next.config.mjs export mode mein headers() nahi deta — static site header nahi bhej sakti, isliye security headers **Amplify** set karta hai.

9.3 Pages (routes)

Route	Kaun	Kya dikhata hai
/	—	/dashboard par redirect
/dashboard	sab	Hero, API status, KPI cards, ingestion/content/department charts, recent documents, admin ke liye AI operations
/chat	sab	Knowledge chat — answer, citations, confidence, model, tokens, cost, latency
/agents	sab	Workflows (5 types) chalana
/documents	sab	Upload, list, filter, delete
/departments	sab	Department-wise documents
/usage	sab (data admin ko)	Usage stats
/feedback	sab	Feedback form
/admin/users	admin	Samjhata hai users Cognito mein bante hain (koi API nahi)
/admin/tenants	admin	Apne tenant ke metrics
/admin/documents	admin	Read-only document list
/admin/metrics	admin	AI metrics
/admin/security, /admin/audit	admin	Events
/admin/deployments	admin	Blue/green explanation + API health

Saare pages "use client" hain — data browser se API call karke aata hai. Server-side data fetching **nahi hai**, isliye static export sambhav hai.

9.4 API client — `src/lib/api.ts`

```
BASE_URL = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000"
request(path) → headers: X-Correlation-ID (naya har request), Authorization: Bearer <token>
    → fetch(BASE_URL + path)
    → network fail: "Could not reach the API. Is the backend running?"
    → 401: token clear
```

`NEXT_PUBLIC_API_URL` :

Environment	Value	Kaun set karta hai
Local	unset → <code>http://localhost:8000</code>	default
AWS Amplify	<code>https://<cloudfront-domain></code>	Terraform (Amplify env var)

Ye value **build time par bundle mein bake** hoti hai. Badalne ke baad **rebuild** chahiye. `amplify.yml` build ko rok deta hai agar value `https://` se shuru nahi hoti (mixed content se bachne ke liye).

9.5 Loading / empty / error states

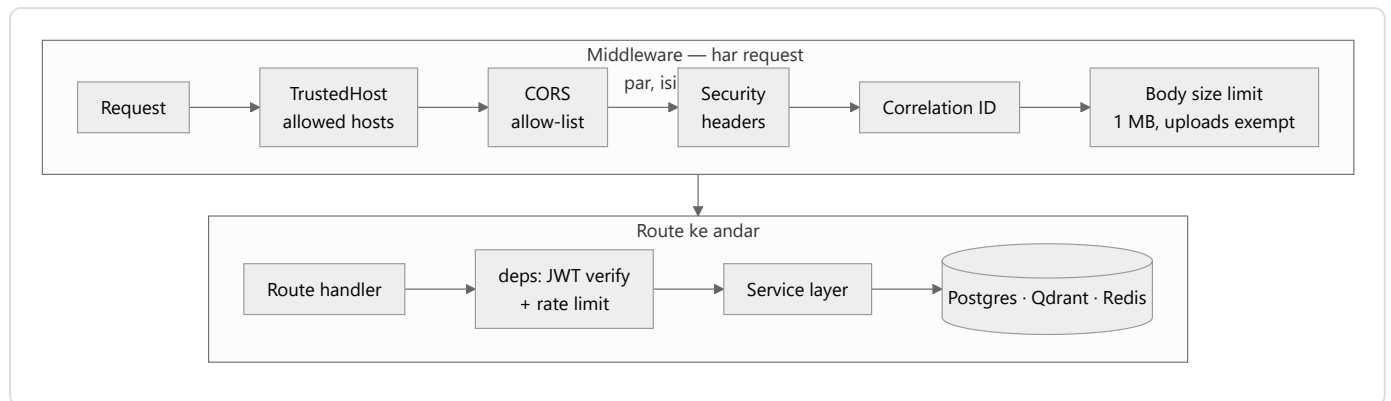
`hooks/useAsync.ts` har data call ko `loading / error / data / reload` deta hai. UI mein `Skeleton`, `EmptyState`, `ErrorState` ("Something went wrong" + Try again) components hain.

9.6 Frontend → backend path

Browser → Amplify (HTML/JS) → [JS fetch] → CloudFront (HTTPS) → ALB (HTTP) → ECS api container

10. Backend — FastAPI Andar Se

10.1 Request ka safar (har request)



10.2 Endpoints (sab `/api/v1` ke neeche)

Method + path	Auth	Kaam
GET <code>/health</code>	none	Liveness — <code>{"status": "ok", "environment": "...", "components": []}</code>
GET <code>/health/ready</code>	none	Postgres + Redis + Qdrant check; koi down → 503
GET <code>/me</code>	user	Token ka principal
POST <code>/documents</code>	user (upload limit)	Upload
GET <code>/documents</code>	user	List (limit ≤ 200)
GET <code>/documents/{id}</code> · <code>/status</code> · <code>/download</code>	user	Detail, ingestion job, presigned URL
DELETE <code>/documents/{id}</code>	owner/admin	Soft delete + Qdrant points + S3 object
POST <code>/chat</code>	user	RAG
POST <code>/feedback</code>	user	Feedback
GET <code>/agents/workflows</code> · POST <code>/agents/run</code>	user (10/min)	Agentic workflows
GET <code>/admin/metrics</code> · <code>/admin/audit</code> · <code>/admin/security</code>	admin	Operations

OpenAPI docs: `http://localhost:8000/docs` — **sirf dev mein** (`docs_url` dev ke bahar `None`).

10.3 Error format

Har error: `{"error": {"code": "...", "message": "..."}, "correlation_id": "...}` — stack trace ya SQL kabhi client ko nahi jaata (`core/exceptions.py`).

11. Database — PostgreSQL

11.1 Technology

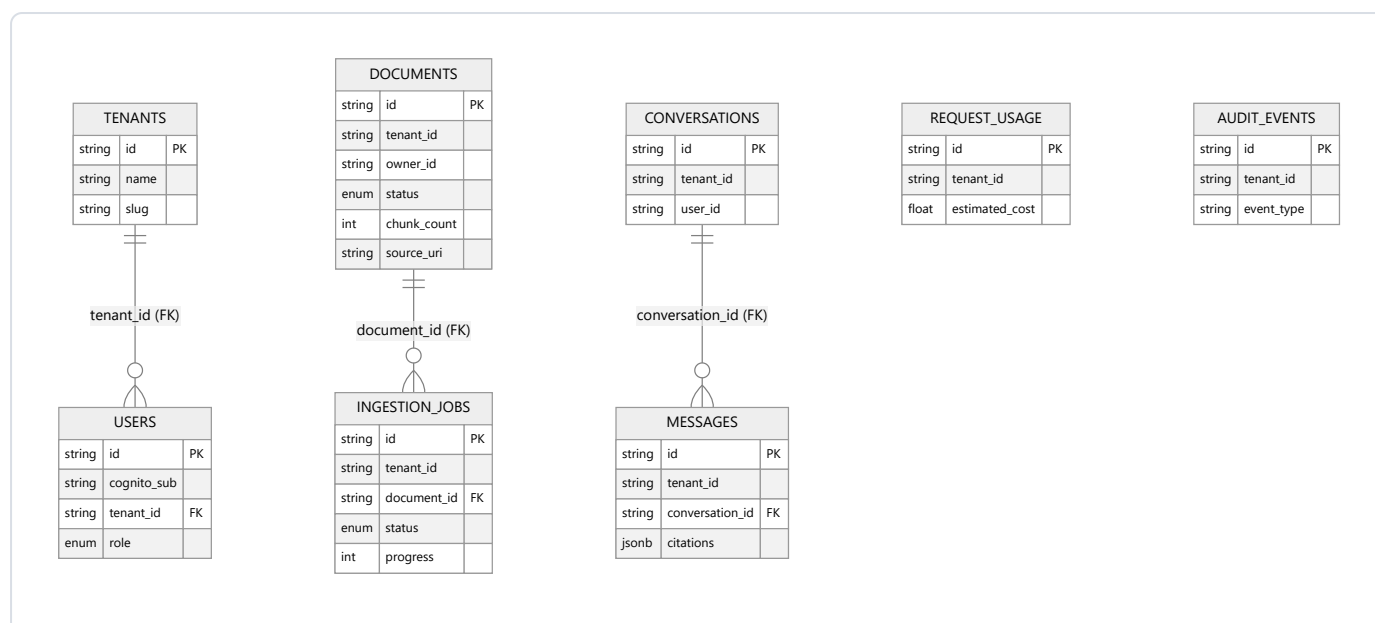
PostgreSQL 16, async SQLAlchemy 2.0 + `asyncpg`, migrations **Alembic**.

- **Local:** Docker Compose container `postgres:16-alpine` (volume ke saath, data bachta hai).
- **AWS: Amazon RDS** — `ekba-dev-postgres`, `db.t4g.micro`, 20 GB gp3, single-AZ, encrypted, private subnets, SG sirf task SG se. Pehle ye ECS sidecar tha (task restart = khali DB); **ab data bachta hai** — task replace, naya deploy, sab ke baad. `destroy.sh` snapshot leta hai aur agla `deploy.sh` usi se restore karta hai.
- **Connection:** container startup par `DATABASE_URL` khud banta hai — `DB_HOST/DB_PORT/DB_NAME/DB_USER` (plain env) + `DB_PASSWORD` (Secrets Manager) + `?ssl=require`. Terraform RDS ka password **write-only** argument se deta hai, isliye password kabhi state ya plan mein nahi aata.
- **Migrations:** har task start par `alembic upgrade head` (schema sahi ho to no-op). Blue aur green **ek hi database** share karte hain, isliye migrations backward-compatible honi chahiye (Chapter 19.3).

11.2 Tables (10)

Table	Kya hai	Tenant column
<code>tenants</code>	Company (id, name, slug)	— (id khud tenant)
<code>users</code>	User (cognito_sub, tenant_id, email, role, department)	✓ FK
<code>documents</code>	File metadata, status, chunk_count, S3 <code>source_uri</code>	✓
<code>ingestion_jobs</code>	Processing job (status, progress, error)	✓
<code>conversations</code>	Chat thread	✓
<code>messages</code>	User + assistant messages, citations, model, confidence	✓
<code>request_usage</code>	Har AI call: tokens, cost, latency, cache_hit	✓
<code>user_feedback</code>	Rating (-1/1), reason, comment	✓
<code>prompt_releases</code>	Prompt versions	— (runtime par use nahi hota)
<code>audit_events</code>	Security/audit events (reason codes, payload kabhi nahi)	nullable

11.3 ER diagram (simplified)



Dhyaan do: `documents.tenant_id` ka `tenants` par foreign key **nahi** hai — ye sirf indexed column hai. Chunks database mein **nahi**, Qdrant mein hain.

11.4 Indexes (tenant filter fast rakhne ke liye)

`ix_documents_tenant_status`, `ix_documents_tenant_department`, `ix_jobs_tenant_status`, `ix_conversations_tenant_user`, `ix_messages_tenant_conv`, `ix_usage_tenant_created`, `ix_audit_tenant_created`, `ix_audit_type`, `ix_feedback_tenant`, `uq_users_tenant_email`.

11.5 Migrations

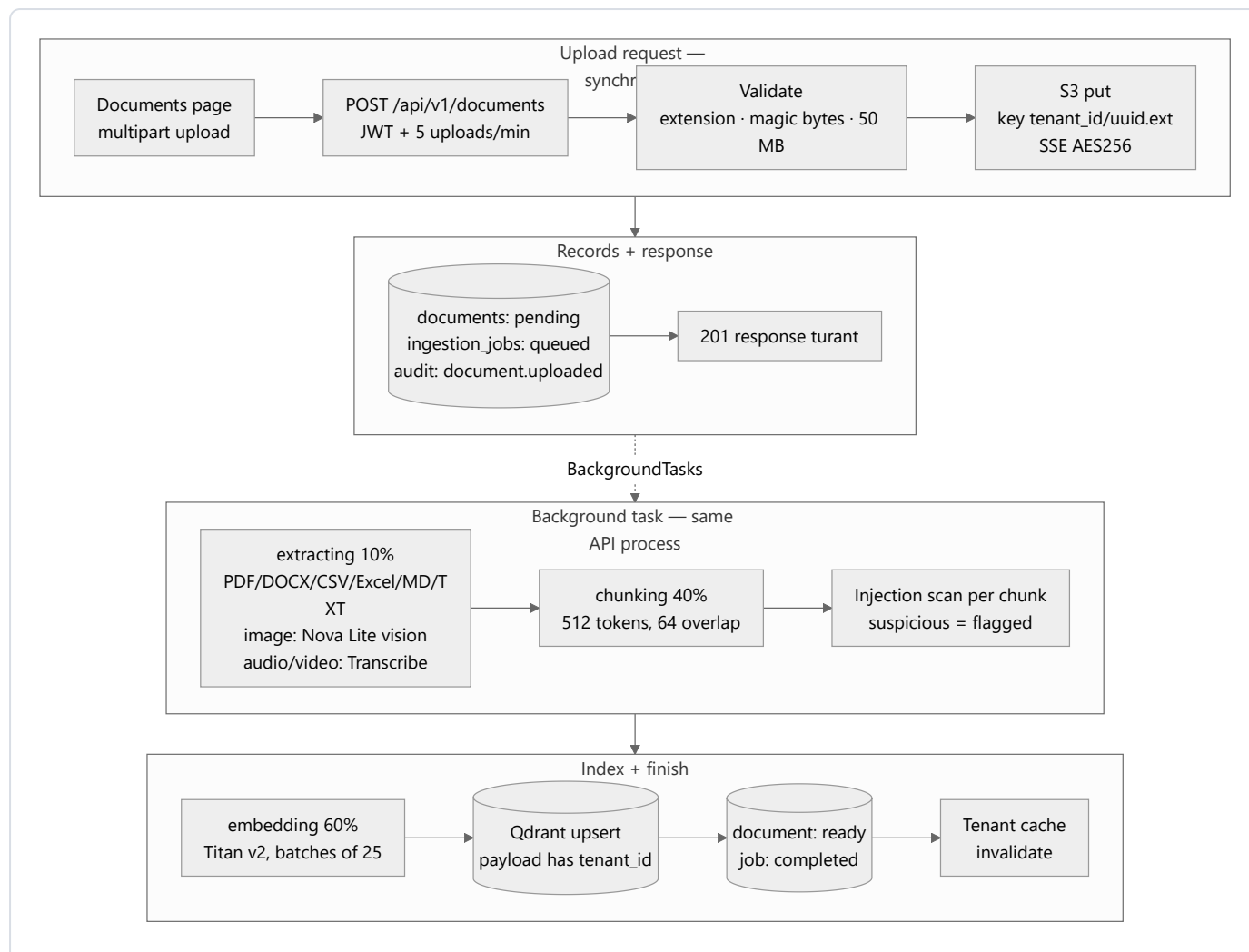
Ek hi migration: `alembic/versions/0001_initial_schema.py`.

```
cd backend
alembic upgrade head      # schema banao/update
alembic current           # kaunsa revision laga hai
```

Naya column: `models.py` edit → `alembic revision --autogenerate -m "..."` → generated file review → `alembic upgrade head` → tests. **Applied migration kabhi edit mat karo.**

12. Document Ingestion Flow

12.1 Diagram



12.2 Har stage

Stage	Input	Kya hota hai	Output	File
Validate	File	Extension allow-list, magic bytes, 50 MB cap, filename sanitize	Safe bytes	<code>security/files.py</code>
Store	Bytes	S3 (local: MinIO) mein <code><tenant_id>/<uuid>.<ext></code>	<code>source_uri</code>	<code>storage.py</code>
Record	—	<code>documents</code> (pending) + <code>ingestion_jobs</code> (queued) + audit	IDs	<code>api/v1/documents.py</code>
Extract	Bytes	Type ke hisaab se extractor	Text blocks (page/section ke saath)	<code>ingestion/extractors.py</code>
Chunk	Blocks	Paragraph → sentence boundaries, overlap	Chunks	<code>ingestion/chunker.py</code>

Stage	Input	Kya hota hai	Output	File
Scan	Chunk text	Hidden instructions dhoondhna (retrieval-poisoning defence)	suspicious flag	security/injection.py → scan_content
Embed	Chunks	25-25 ke batch	1024-dim vectors	ai/provider.py
Index	Vectors + payload	Qdrant upsert (tenant_id ke bina refuse)	Points	vector.py
Finish	—	Document ready, job completed, cache invalidate	—	ingestion/pipeline.py

Supported types (`files.py`): pdf, txt, md, doc, docx, csv, xlsx, xls, png, jpg, jpeg, webp, gif, mp3, wav, m4a, flac, ogg, mp4, mov, webm.

Likely limitation (test nahi kiya): `.doc` aur `.xls` allow-list mein hain, lekin extractor `python-docx` (sirf `.docx`) aur `openpyxl` (sirf `.xlsx`) use karta hai. Purane binary `.doc` / `.xls` parse fail hone ki sambhavna hai. `requirements.txt` ke comment "DOC / DOCX", "XLSX / XLS" isse match nahi karte.

12.3 Status kaise badalta hai

Document: `pending` → `processing` → `ready` (ya `failed`), delete par `deleted`. Job: `queued` → `extracting` → `chunking` → `embedding` → `completed` (ya `failed` + `error_code`). `transcribing` status enum mein hai.

Failure par: document `failed`, job `failed` with `error_code` (exception ka naam), audit event record hota hai. Background task crash kabhi silently nahi hota — `ingestion_task_crashed` log hota hai.

12.4 Manually check karna

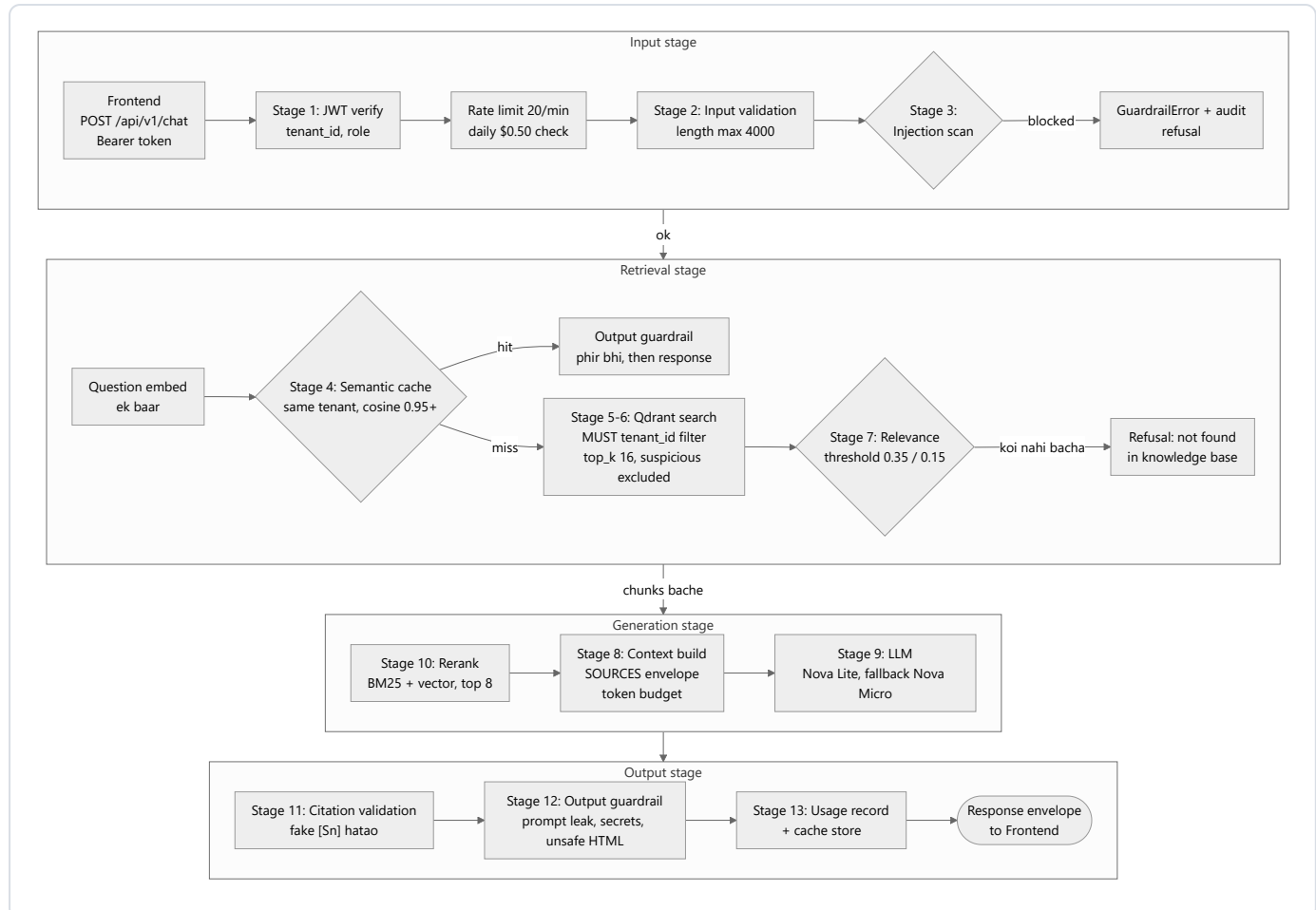
```
# API se (token ke saath)
curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/documents/<DOC_ID>/status

# Local DB se
docker exec -it ekba-dev-postgres-1 psql -U ekba -d ekba \
-c "select name,status,chunk_count from documents order by created_at desc limit 10;"
```

13. RAG Query Flow — Sabse Important Chapter

13.1 Example

User (northwind) poochta hai: **"What is the domestic hotel reimbursement limit?"**



13.2 Har step — Hinglish mein

#	Step	Input → Output	File
1	Authentication	Token → <code>RequestContext(user_id, tenant_id, role)</code>	<code>api/deps.py</code> , <code>core/auth.py</code>
—	Daily ceiling	Aaj ka <code>request_usage</code> sum ≥ \$0.50 → "reached today's AI usage limit"	<code>api/v1/chat.py</code>
2	Input validation	Sawaal → cleaned sawaal (max 4000 chars, control chars reject)	<code>security/injection.py</code> → <code>validate_question</code>
3	Injection scan	Sawaal → safe / blocked. Pattern families: <code>instruction_override</code> , <code>system_prompt_extraction</code> , <code>role_manipulation</code> , <code>guardrail_bypass</code> , <code>exfiltration</code> , <code>delimiter_injection</code> , <code>credential_probe</code> , <code>embedded_directive</code> , <code>hidden_instruction_marker</code> , <code>prompt_leak_bait</code> (+ invisible chars, encoded blobs)	<code>injection.py</code>
—	Embed	Sawaal → 1024-dim vector (cache aur search dono ke liye ek hi call)	<code>ai/provider.py</code>

#	Step	Input → Output	File
4	Semantic cache	Vector → cached answer agar isi tenant ka koi sawaal cosine ≥ 0.95 ho	rag/cache.py
5–6	Tenant-filtered retrieval	Vector → top 16 chunks, <code>must: tenant_id == ctx.tenant_id</code> , optional document_ids/department filter, <code>suspicious</code> chunks bahar	vector.py → search
7	Relevance threshold	Score < 0.35 (local 0.15) wale hatao; kuch na bache → "not found" answer, model call nahi	rag/pipeline.py
10	Rerank	BM25-style lexical score (weight 0.35) + vector score → top 8	rag/rerank.py
8	Context construction	Chunks → <code><<<BEGIN_SOURCES>>> ... [S1] ... <<<END_SOURCES>>></code> data envelope, budget $\sim 8192 \times 4$ chars	rag/prompts.py
9	Model call / routing	System prompt + context + sawaal → answer text. Primary fail → fallback (logged, <code>model_used</code> mein dikhta)	ai/provider.py
11	Citation validation	<code>[Sn]</code> markers → sirf retrieved chunks map hote hain; invented hata diye, foreign-tenant citation drop	rag/guardrails.py
12	Output guardrail	System prompt echo, AWS key / private key / JWT, <code><script></code> , <code>javascript:</code> ... → block	rag/guardrails.py
—	Confidence	$0.6 \times \text{avg}(\text{top-3 scores}) + 0.4 \times \text{citation coverage} - 0.2 \times \text{invented}$	guardrails.compute_confidence
13	Usage tracking	Tokens, cost, latency → <code>request_usage</code> ; clean answer + citations → cache	repositories.record_usage
14	Language	System prompt mein instruction	prompts.py

Phir `chat.py` conversation + dono messages (user, assistant) DB mein save karta hai.

13.3 Response envelope (frontend ko kya milta hai)

```
answer, citations[], retrieved_chunks[] (280-char preview), model_used, input_tokens, output_tokens,
estimated_cost, latency_ms, cache_hit, tenant_id, confidence, correlation_id, conversation_id
```

13.4 Hallucination kaise kam hota hai

1. Model ko sirf retrieved context diya jaata hai (system prompt: "grounded in provided context with citations").
2. Relevance threshold — kuch relevant nahi to model call hi nahi, seedha "not found".
3. Retrieved text **data envelope** mein — document ke andar likhi "instructions" follow nahi hoti.
4. Citation validation — banaya hua source number hata diya jaata hai aur confidence gir jaata hai.
5. Low temperature (`0.1`).

13.5 Local mode vs AWS mode

	Local (<code>AI_PROVIDER=local</code>)	AWS (<code>AI_PROVIDER=bedrock</code>)
Embeddings	Hashed bag-of-words (deterministic, \$0)	Titan Text Embeddings V2
Chat	Extractive stub — context ki lines <code>[Sn]</code> ke saath quote karta hai, <code>model_used = "local-dev-stub"</code>	Nova Lite (fallback Nova Micro)

	Local (AI_PROVIDER=local)	AWS (AI_PROVIDER=bedrock)
Threshold	0.15	0.35
Cost	\$0	Per token (Chapter 14)

13.6 Agentic workflows (LangGraph)

POST /api/v1/agents/run — workflows: policy_comparison , summarization , cross_document_analysis , knowledge_extraction , report_generation . Graph: plan → retrieve → analyze → synthesize → cite . Bounds: **MAX_STEPS = 12, timeout 120 s**; timeout par partial result (partial: true). Retrieval same tenant filter use karta hai. Input injection scan yahan bhi.

14. AI / LLM / Embeddings

14.1 Models

Role	Model ID	Status
Chat primary	<code>us.amazon.nova-lite-v1:0</code>	Configured (AWS quota 0 → abhi blocked)
Chat fallback	<code>us.amazon.nova-micro-v1:0</code> (text-only)	Configured, fallback
Vision (images)	<code>us.amazon.nova-lite-v1:0</code>	Configured
Embeddings	<code>amazon.titan-embed-text-v2:0</code> , 1024 dims, normalized	Configured
Speech	Amazon Transcribe	Configured
Local dev	<code>local-dev-stub</code> (chat + embeddings)	Active locally
OpenAI / Azure / Vertex / Ollama	—	Implemented nahi hai (rules ke hisaab se allowed bhi nahi)
<code>gpt-oss</code> models	—	Plan mein the; <code>us-west-2</code> mein available nahi the → Nova se replace

Model IDs **configuration** hain (`.env` / Terraform vars) — badalne ke liye code change nahi. Nova ko **inference-profile ID** (`us.` prefix) chahiye; bare ID par Bedrock `ValidationException` deta hai. Titan bina prefix ke.

14.2 Pricing table (code mein, `ai/registry.py`, per 1M tokens)

Model	Input \$	Output \$
nova-micro	0.035	0.14
nova-lite	0.06	0.24
titan-embed-text-v2	0.02	—

14.3 Flow

```

Question → Titan embed → vector → Qdrant search → chunks → Nova Lite → answer
Document → extract → chunks → Titan embed (batch 25) → vectors → Qdrant
Image    → Nova Lite vision (describe) → text → chunks → ...
Audio/Video → Transcribe → transcript → chunks → ...

```

Operational limitation (2026-09-11): Bedrock quotas 0. ECS logs mein aata hai `bedrock_embed_failed ... ThrottlingException` → `UpstreamError: The embedding service is unavailable.` → `seed failed - continuing without demo data`. API healthy rehta hai; chat aur ingestion quota milne tak nahi challenge. Quota check:

```
aws service-quotas list-service-quotas --service-code bedrock --region us-west-2 \
  --query "Quotas[?contains(QuotaName,'Titan Text Embeddings V2') || contains(QuotaName,'Nova Lite')].
[QuotaName,Value]" --output table
```

15. Qdrant — Vector Database

Vector database kya hai? Normal DB exact match dhoondhta hai; vector DB **meaning** se multi-julti cheez dhoondhta hai. **Embedding** = text ka numbers mein roop (yahan 1024 numbers ki list). Milte-julte meaning wale text ke vectors paas-paas hote hain. **Similarity search** = sawaal ke vector ke sabse paas wale chunk vectors dhoondhna (yahan **cosine** distance).

Setting	Value
Image	<code>qdrant/qdrant:v1.12.4</code>
Collection	<code>ekba_chunks</code> (<code>QDRANT_COLLECTION</code>)
Dimension	1024, distance COSINE
Payload index	<code>tenant_id</code> (KEYWORD) — mandatory filter ke liye
Startup	<code>ensure_collection()</code> — collection na ho to banata hai; dimension mismatch par error ("create a new collection and re-index")

Har point ka payload: `tenant_id, document_id, chunk_id, document_name, page_number, section, source_uri, owner_id, document_version, created_by, created_at, text, modality, department, suspicious`.

Tenant filter: `vector.search` khud `must: tenant_id == ctx.tenant_id` banata hai — koi caller filter override nahi kar sakta. `upsert_chunks` bina `tenant_id` wala chunk likhne se mana karta hai.

"Leave policy" PDF → 4 chunks → 4 vectors (payload: `tenant_id=northwind...`) → Qdrant
 "annual leave carry over?" → vector → cosine search (northwind only) → best chunk → answer

Local dekhna: `http://localhost:6333/dashboard` (Qdrant UI) ya `curl -s http://localhost:6333/collections/ekba_chunks` (`points_count` dekho — seed ke baad 41).

16. Redis — Cache aur Rate Limit

Redis `redis:7-alpine` , **persistence off** (`--save "" --appendonly no`) — restart par sab khali.

Use	Key pattern	TTL
Rate limit	<code>ekba:r1:<bucket>:<tenant_id>:<user_id></code>	60 s window
Semantic cache index	<code>ekba:sc:idx:<tenant_id></code> (last 50 question digests)	3600 s
Semantic cache entry	<code>ekba:sc:e:<tenant_id>:<sha256[:32]></code> (answer, citations, vector, model, confidence)	3600 s

Cache hit = isi tenant ka pehle ka sawaal cosine ≥ 0.95 . **Miss** = poori RAG pipeline. Document upload/delete par us tenant ka cache **invalidate**.

Redis down ho to? Cache ke errors pakde jaate hain (`# cache must never break the request path`) — chat chalta rahega bina cache ke. Rate limiting Redis par depend karta hai; `/health/ready` Redis ko unhealthy dikhayega.

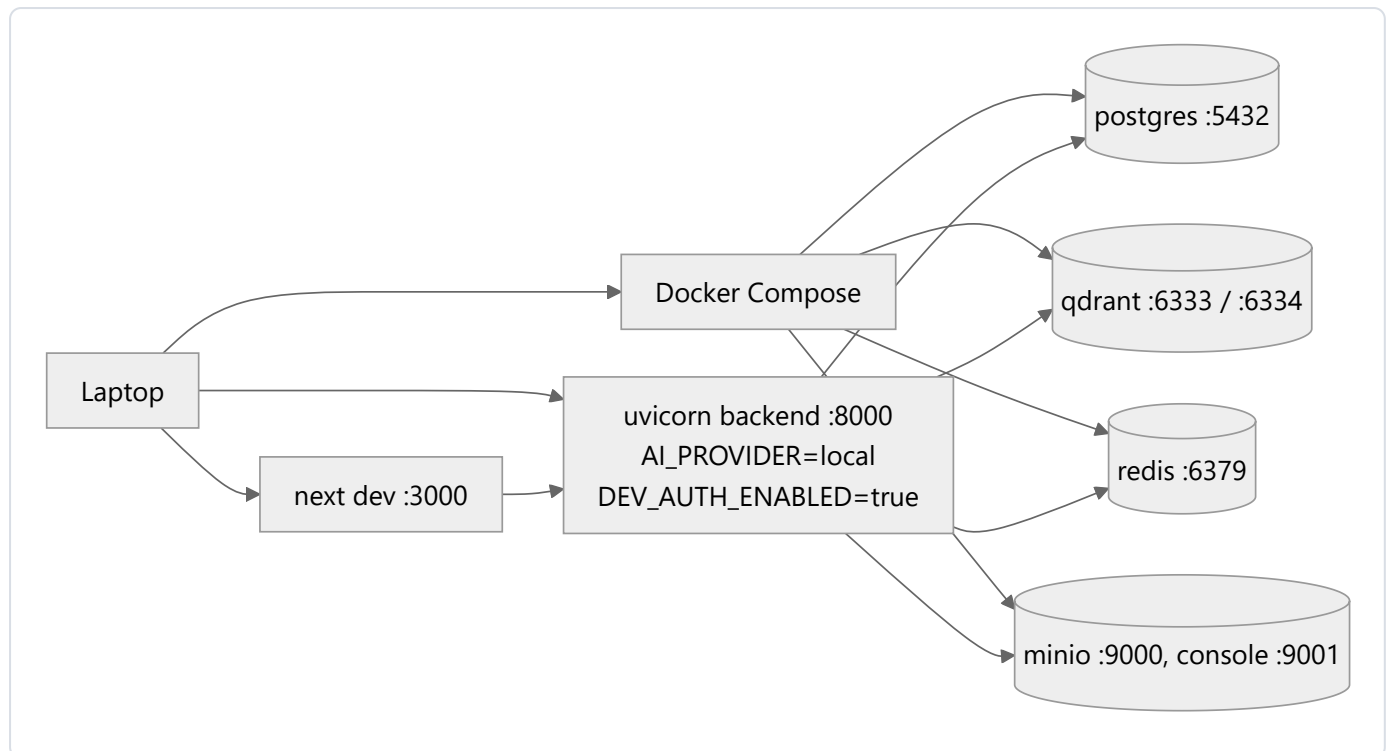
17. Data Storage Map — Kya Kahan Rehta Hai

Data	Kahan	Kyun	Kaise dekhain
Original files	S3 (local: MinIO) <code><tenant>/<uuid>.<ext></code>	Persistent, encrypted	<code>aws s3 ls s3://<BUCKET>/<tenant>/</code> · MinIO console <code>http://localhost:9001</code>
Document metadata, jobs, conversations, messages, usage, feedback, audit	PostgreSQL — local: container; AWS: RDS (data bachta hai)	Relational	Local: <code>docker exec -it ekba-dev-postgres-1 psql -U ekba -d ekba</code> ; AWS: sirf API/logs se
Chunks + embeddings	Qdrant <code>ekba_chunks</code>	Vector search	<code>http://localhost:6333/dashboard</code>
Cache + rate limit counters	Redis	Speed / abuse control	<code>docker exec -it ekba-dev-redis-1 redis-cli --scan --pattern 'ekba:*'</code>
User identity, tenant, role	Cognito (AWS) / dev token (local)	Auth	Cognito console / <code>list-users</code>
Secrets	Secrets Manager (AWS) / <code>.env</code> (local, git-ignored)	Security	Sirf naam dekho, value nahi
App logs	CloudWatch <code>/ekba/dev/service</code> (AWS) / terminal (local)	Debugging	<code>aws logs tail</code>
Terraform state	S3 <code>ekba-tfstate-<ACCOUNT_ID></code> + DynamoDB lock	Infra record	<code>terraform state list</code> (kabhi haath se edit nahi)
Deployment reports	<code>docs/reports/*.md</code> (local files)	Audit trail	File kholo (account ID hota hai — commit mat karo)

AWS par PostgreSQL RDS hai (private subnets) aur Qdrant/Redis task ke andar containers hain. Dono tak direct public raasta nahi: RDS ka SG sirf task SG se allow karta hai, aur task ka SG sirf ALB se. AWS par data dekhna ho to **API endpoints** (admin metrics, documents list) ya CloudWatch logs use karo. ECS Exec configure **nahi** hai.

18. Local Development — Laptop Par Poora Project

18.1 Picture



Local mein \$0 — koi AWS nahi, local AI stub, local dev tokens.

18.2 Sabse aasaan raasta — ek command

```

./scripts/setup-local.sh      # Git Bash / macOS / Linux
.\scripts\setup-local.ps1    # Windows PowerShell
  
```

Ye 7 steps karta hai: prerequisites check → `.env` banana (generated secrets) → backend deps → containers → migrations → seed → frontend deps. Dobra chalana safe hai.

18.3 Haath se, step by step (RUNBOOK Part A se verified)

Step 1 — `.env` banao

```
cp .env.example .env
```

Chaar values set karo: `POSTGRES_PASSWORD`, `DATABASE_URL` (same password), `DEV_AUTH_SECRET` (32+ chars), `S3_BUCKET=ekba-dev-documents`. Local ke liye ye rehne do: `ENVIRONMENT=dev`, `AI_PROVIDER=local`, `DEV_AUTH_ENABLED=true`, `S3_ENDPOINT_URL=http://localhost:9000`.

Step 2 — Containers

```
docker compose --env-file .env -f infra/docker/docker-compose.yml up -d postgres qdrant redis minio minio-init
docker compose --env-file .env -f infra/docker/docker-compose.yml ps
```

`--env-file .env` **zaroori hai** — compose file `infra/docker/` mein hai, bina iske root `.env` nahi padha jaata aur password mismatch hota hai.

Step 3 — Backend install

```
cd backend
python -m venv .venv
source .venv/Scripts/activate          # Git Bash   (PowerShell: .\.venv\Scripts\Activate.ps1)
pip install -r requirements.txt -r requirements-dev.txt
```

Step 4 — Tables → `alembic upgrade head` **Step 5 — Demo data** → `python -m seeds.seed` (output mein `chunks_indexed: 41`) **Step 6 — Backend** → `uvicorn app.main:app --reload --port 8000` **Step 7 — Frontend** (naya terminal) → `cd frontend && npm install && npm run dev` **Step 8 — Token** → `cd backend && python -m seeds.dev_token` (admin: `--role admin --user seed-admin-a`) **Step 9 — Kholo** → `http://localhost:3000`, token paste karo.

18.4 Check, logs, docs

```
curl http://localhost:8000/api/v1/health          # liveness
curl http://localhost:8000/api/v1/health/ready    # postgres + redis + qdrant
# API docs (sirf dev): http://localhost:8000/docs
docker compose --env-file .env -f infra/docker/docker-compose.yml logs -f qdrant
```

Backend logs uvicorn wale terminal mein aate hain (JSON lines, `correlation_id` ke saath).

18.5 Stop / restart

```
# Backend / frontend: us terminal mein Ctrl+C
docker compose --env-file .env -f infra/docker/docker-compose.yml stop    # containers roko, data rahe
docker compose --env-file .env -f infra/docker/docker-compose.yml down    # containers hatao, volumes
rahein
docker compose --env-file .env -f infra/docker/docker-compose.yml down -v # DATA BHI DELETE
```

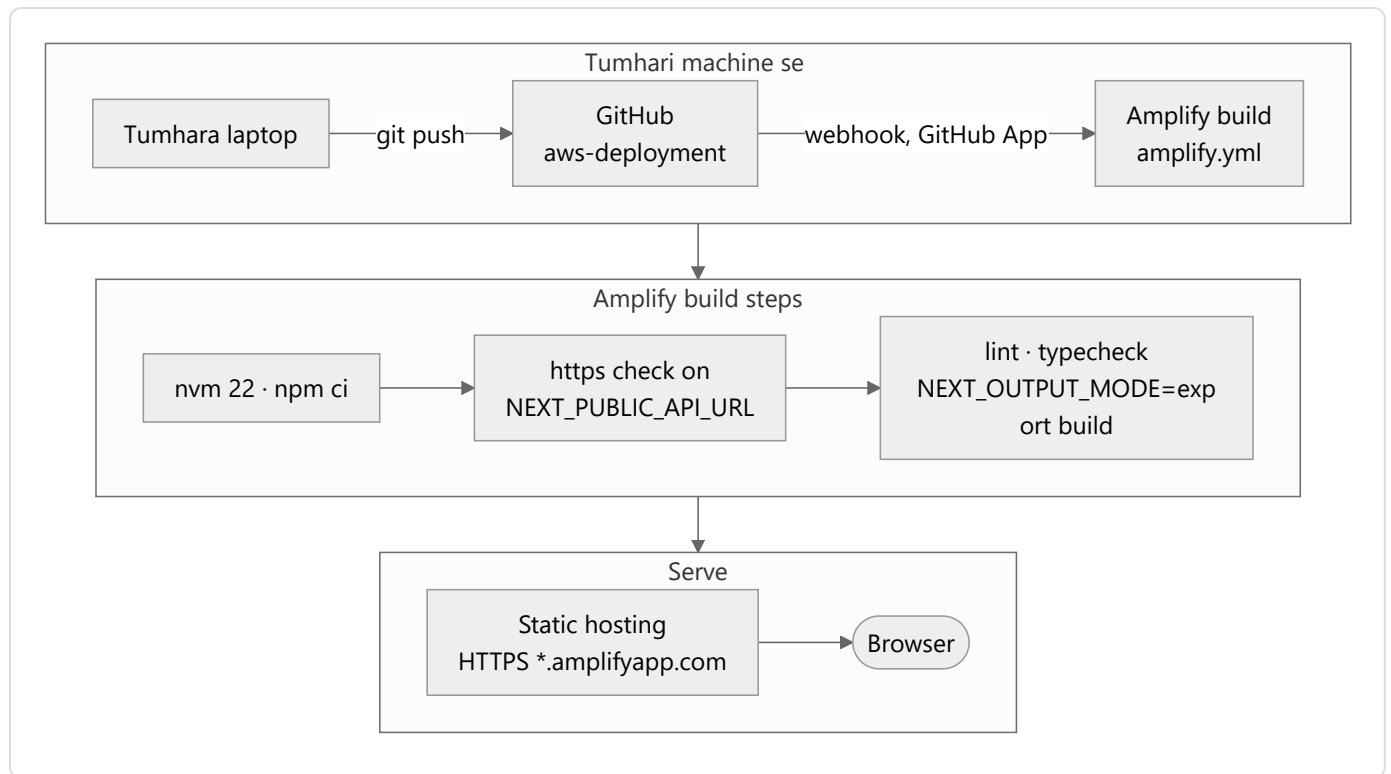
Port 8000 atka ho (PowerShell):

```
Get-NetTCPConnection -LocalPort 8000 -State Listen | ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }
```

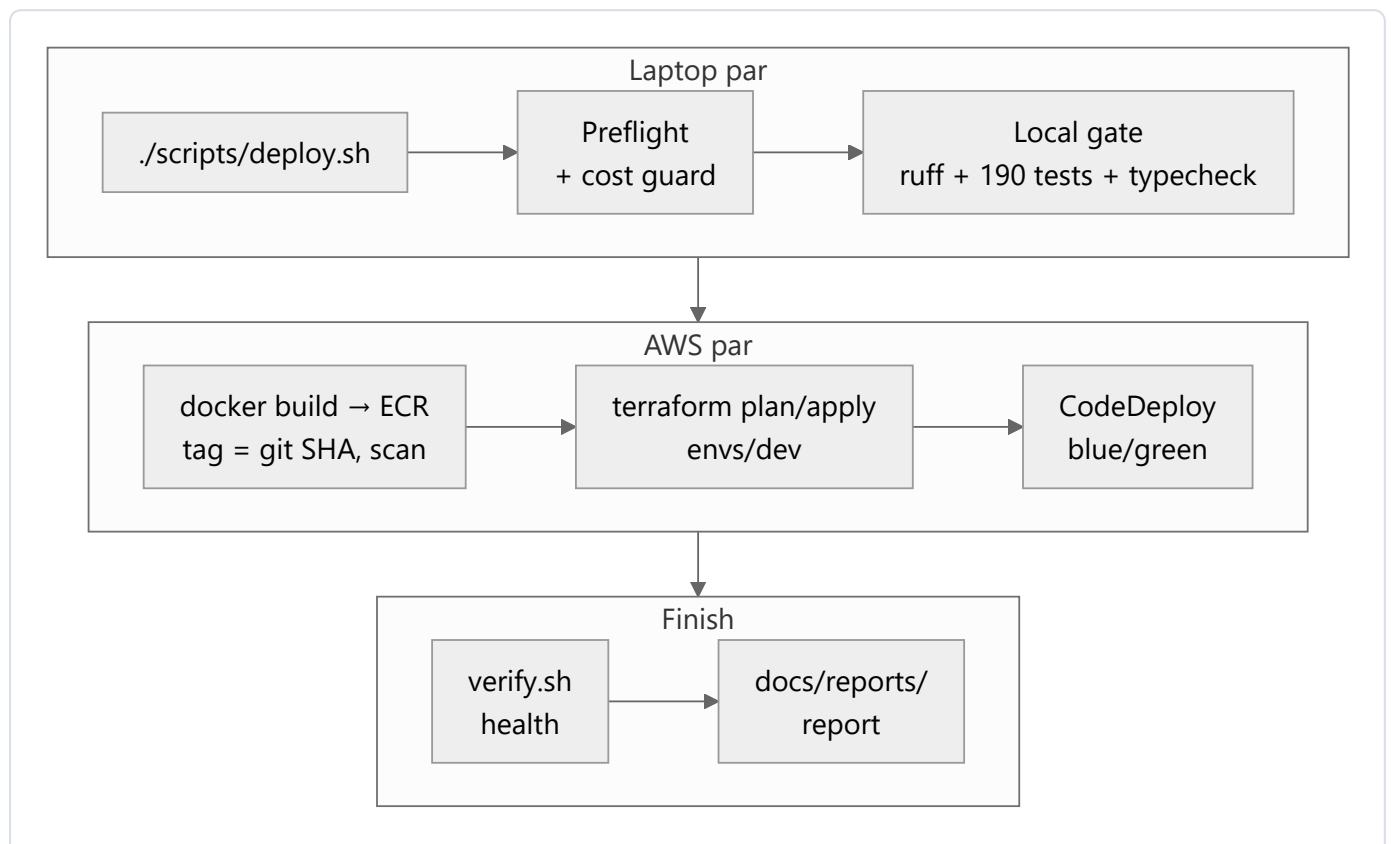
Dev server chalte hue `npm run build` **mat chalo**. Dono `frontend/.next` share karte hain — build dev server ki files overwrite kar deta hai aur pages load hona band ho jaata hai. Pehle dev server roko.

19. AWS Production Flow

19.1 Frontend

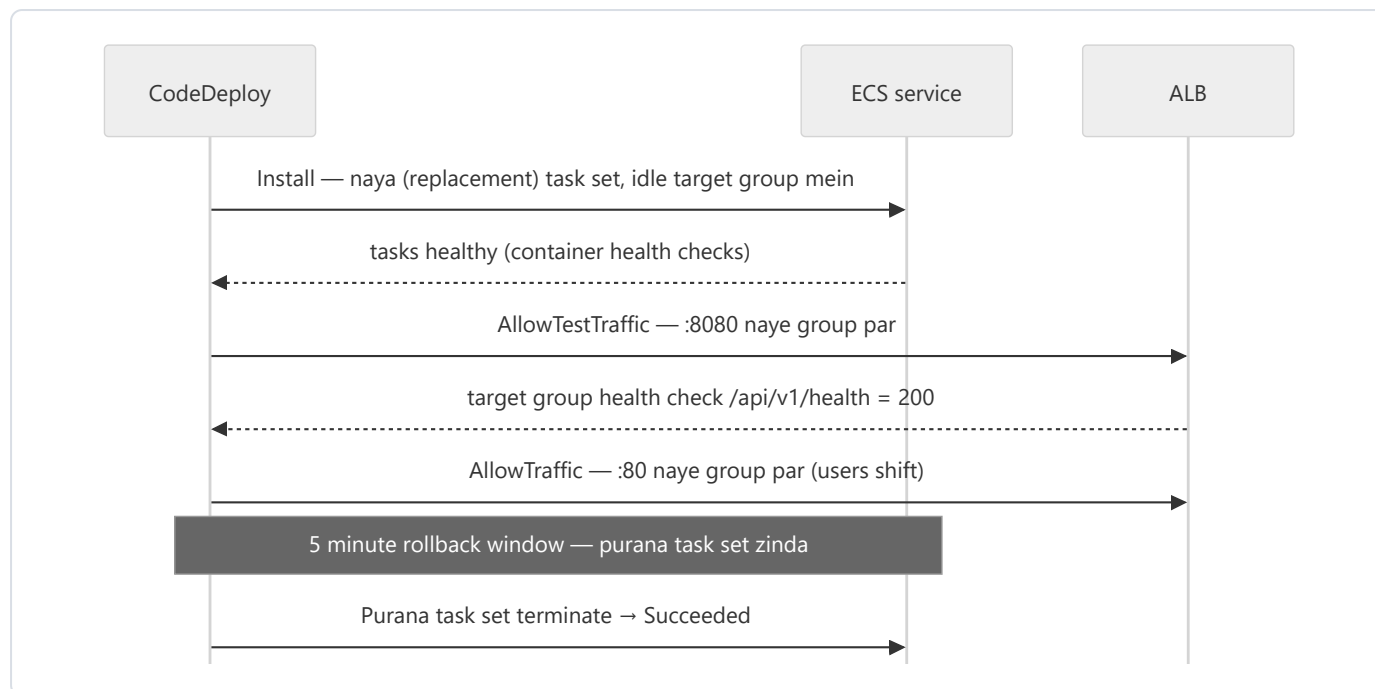


19.2 Backend



Backend deploy **tumhare laptop se** hota hai — git push se automatically **nahi** hota.

19.3 Blue/green — kaise kaam karta hai



Har `deploy.sh` ek poora blue/green karta hai (~9 min CodeDeploy, ~11–12 min total), chahe kuch na badla ho. Target groups `ekba-dev-blue` ↔ `ekba-dev-green` har baar roles badalte hain.

Honest note: AppSpec mein lifecycle hooks **nahi** hain — green ka test sirf container health checks + ALB health check (`/api/v1/health` , liveness) hai. DB/Qdrant/Redis/RAG smoke test deployment ke dauran nahi hota (rules mein required hai, implemented nahi).

20. "Mujhe Code Change Karna Hai — Ab Kya Karu?"

20.1 Backend change

```
git status # 1. kya badla hai
cd backend && source .venv/Scripts/activate
ruff check app tests seeds && ruff format --check app tests seeds # 2. lint
ENVIRONMENT=dev AI_PROVIDER=local DEV_AUTH_ENABLED=true \
  pytest tests/unit tests/integration tests/security tests/evaluation -q # 3. tests
cd .. && git diff # 4. review
git add <files> && git commit -m "..." # 5. commit (image tag = commit SHA)
git push origin <branch> # 6. push (backup + CI agar main)
curl -s https://checkip.amazonaws.com # 7. IP == allowed_cidrs? (Chapter 28)
./scripts/deploy.sh # 8. deploy
```

`deploy.sh` khud karta hai: cost guard → local gate (dobara) → Docker build → ECR push (immutable SHA tag; image pehle se ho to reuse) → scan → `terraform fmt/validate/plan` → **destroy/delete dikhe to typed confirmation** `I REVIEWED THIS PLAN` → apply → CodeDeploy → `verify.sh` → report.

`backend/` ya `backend.Dockerfile` mein uncommitted changes hon to `deploy.sh` ruk jaata hai — image ka tag commit SHA hai, isliye image mein exactly wahi commit hona chahiye. Pehle commit karo.

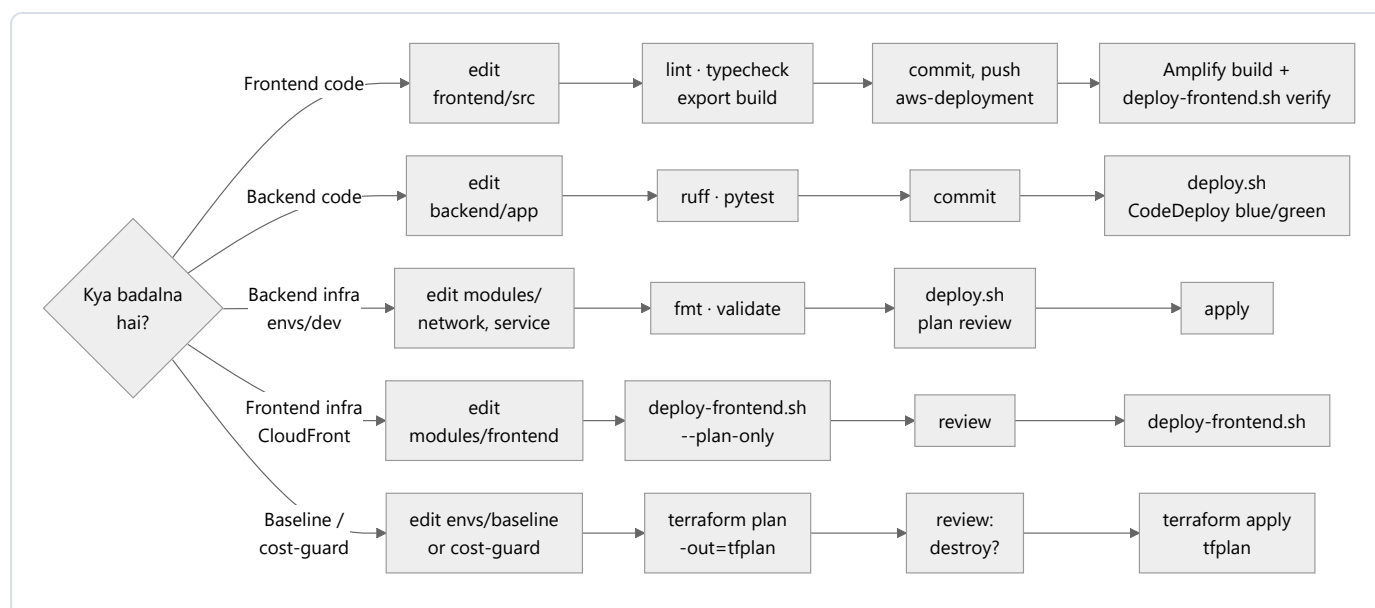
Terraform change bhi ho (jaise naya env var): wahi `deploy.sh` — plan mein dikhega (task definition "must be replaced" normal hai, Chapter 24.3).

20.2 Frontend change

```
cd frontend
npm run lint && npm run typecheck
NEXT_OUTPUT_MODE=export NEXT_PUBLIC_API_URL=https://example.invalid npm run build # Amplify jaisa build
cd .. && git add frontend/... && git commit -m "..."
git push origin aws-deployment # ← yahi deploy hai: Amplify auto-build (~4 min)
./scripts/deploy-frontend.sh # optional: 12 checks se verify
```

Terraform **zaroori nahi** — sirf code change hai to push kaafi hai. Terraform tab chahiye jab Amplify settings/CloudFront badalne hon (`deploy-frontend.sh` wo karta hai).

20.3 Decision tree



21. Branch Strategy — Asli Setup

Branch	Abhi kahan	Kya trigger karta hai
aws-deployment	15c2a00 (latest)	Amplify auto-build (frontend). CI nahi chalta
main	dd545c4 (peeche hai)	CI (ci.yml): push to main / develop , PR to main)
develop	exist nahi karta	CI trigger list mein hai

Backend deploy **kisi branch push se nahi** — `./scripts/deploy.sh` jo bhi local commit checked-out hai use deploy karta hai. `deploy.yml` (GitHub Actions) manual hai aur **abhi tak run nahi hua** (Chapter 25).

Tumhara intended workflow: `main` pe kaam → `aws-deployment` mein merge → push → Amplify.

Pehle ek baar sync karna padega: `main` abhi 6 commits peeche hai. Jab tum decide karo:

```
git checkout main
git pull origin main
git merge --ff-only aws-deployment    # fast-forward, koi merge commit nahi
git push origin main
```

Uske baad roz ka flow:

```
git checkout main && git pull
# ... kaam ...
git add <files> && git commit -m "..." && git push origin main    # CI chalega
git checkout aws-deployment && git pull
git merge main                                                    # ya --ff-only
git push origin aws-deployment                                    # Amplify deploy
```

`git add .` se bacho — reports (`docs/reports/2026*.md` , account ID ke saath) untracked padi rehti hain; files naam se add karo.

22. Stop / Start / Destroy — Cheat Sheet

22.1 Stop vs Destroy

	Matlab	Kharcha
STOP	Temporarily band — resource rehta hai	Band cheez ka compute bachta, par ALB jaisi cheezein chalti rehti hain
DESTROY	Resource hi hata do (database ka snapshot pehle ban jaata hai)	<code>envs/dev</code> ka poora ~\$0.11/hr khatam

Project mein AWS "stop" script nahi hai. Supported tareeka `destroy.sh` hai. ECS ko haath se 0 par scale karna (`aws ecs update-service --desired-count 0`) ALB ka kharcha nahi rokta (~\$24/month idle) aur Terraform drift banata hai — cost guard (armed hone par) exactly yahi + ALB delete karta hai.

22.2 Commands

Kaam	Command
Local frontend stop / start	Ctrl+C / <code>cd frontend && npm run dev</code>
Local backend stop / start	Ctrl+C / <code>cd backend && uvicorn app.main:app --reload --port 8000</code>
Local containers stop / start	<code>docker compose --env-file .env -f infra/docker/docker-compose.yml stop / up -d ...</code>
AWS kya chal raha hai	<code>./scripts/cost-check.sh</code>
AWS backend deploy / update	<code>./scripts/deploy.sh</code>
AWS frontend deploy / verify	<code>git push origin aws-deployment / ./scripts/deploy-frontend.sh</code>
AWS backend destroy	<code>./scripts/destroy.sh</code> → type <code>DESTROY ekba-dev</code>
Destroy ke baad wapas	<code>./scripts/deploy.sh</code> phir <code>./scripts/deploy-frontend.sh</code>

22.3 `destroy.sh` kya karta hai

1. Account, region, workspace, state verify
2. Har resource print karta hai jo hatega
3. Plan mein protected cheez (secrets, ECR, state backend, budget, IAM user...) → **abort**
4. Typed confirmation `DESTROY ekba-dev` (sirf `y` nahi)
5. Sirf `envs/dev` state destroy
6. Secrets bache hain confirm + audit report

Bachta hai: baseline (ECR images, S3 documents, Cognito, secrets, budget, audit logs, OIDC role), cost guard, frontend (Amplify + CloudFront), state backend. **Hatta hai:** VPC, subnets, SGs, ALB, target groups, ECS cluster/service/task defs, CodeDeploy app, task IAM roles, service log group, alarms.

22.4 Destroy ke baad redeploy

```
./scripts/deploy.sh          # naya ALB (naya DNS naam); frontend.auto.tfvars se CORS + CloudFront
                              ingress khud lagta hai
./scripts/deploy-frontend.sh # CloudFront origin naye ALB par (in-place), rebuild nahi
```

Frontend aur API URL **same** rehte hain.

22.5 Rollback — galat version live ho gaya to kya karu?

Kya	Kaise	Detail
Frontend — pichla build wapas	<code>./scripts/rollback-frontend.sh</code>	Live build se pehle wala last successful build dobara release karta hai (Amplify job)
Frontend — specific commit	<code>./scripts/rollback-frontend.sh --to <COMMIT_SHA></code>	Us commit ka build
Frontend — rollback permanent karna	<code>git revert <BAD_SHA></code> → <code>git push origin aws-deployment</code>	Agla push branch head ko hi deploy karta hai, isliye bina revert ke rollback agle push par hat jaata hai
Backend — automatic	CodeDeploy khud	Deployment fail (<code>DEPLOYMENT_FAILURE</code>) ya alarm <code>ekba-dev-5xx</code> → traffic purane (blue) task set par. Purana set 5 minute zinda rehta hai. Configured hai, abhi tak exercise nahi hua
Backend — deployment chal raha ho	<code>aws deploy stop-deployment --deployment-id <DEPLOYMENT_ID> --auto-rollback-enabled</code>	Sirf in-flight deployment ke liye
Backend — deploy ho chuka, galat nikla	<code>git revert <BAD_SHA></code> → commit → <code>./scripts/deploy.sh</code>	"Fix forward": naya image, naya blue/green deploy. Recommended tareeka

`./scripts/rollback.sh` **traffic wapas NAHI shift karta**. Uska CodeDeploy step code mein "not yet implemented (Phase 5)" hai — ye sirf `verify.sh` chalata hai aur report likhta hai. Isliye backend rollback = CodeDeploy auto-rollback ya `git revert` + `deploy.sh`.

Purana commit `git checkout` karke `deploy.sh` chalana possible hai (same SHA ka image ECR mein ho to reuse hota hai), lekin us commit ka **Terraform bhi apply hoga** — purana Terraform naye infra se mismatch kar sakta hai. Isse bacho.

AWS frontend "stop" karna: Project mein currently ye implemented nahi hai — Amplify ka koi stop button/script nahi hai aur `destroy.sh` frontend ko touch nahi karta. Zaroorat bhi nahi: idle Amplify + CloudFront ka kharcha ~\$0 hai. Backend destroy ho to site khulti rahegi par "Could not reach the API" dikhayegi. `envs/frontend` par seedha `terraform destroy` project rules ke khilaaf hai (destroy sirf `destroy.sh` se).

23. Docker

Image = packaged app (code + dependencies). **Container** = image ka chalta hua instance. **Port** = bahar se andar ka raasta (`-p 8000:8000`). **Volume** = container ke bahar data (restart par bachta hai). **Network** = containers aapas mein baat karein. **Compose** = kai containers ek file se.

File	Kya
infra/docker/docker-compose.yml	Local: postgres, qdrant, redis, minio, minio-init, + optional <code>api</code> aur <code>web</code>
infra/docker/backend.Dockerfile	python:3.12-slim , non-root user <code>ekba</code> (uid 10001), port 8000, HEALTHCHECK, <code>uvicorn</code>
infra/docker/frontend.Dockerfile	node:22-alpine , Next.js standalone (Amplify ise use nahi karta)

```
docker ps                # "process status" – chal rahe containers
docker ps -a             # ruke hue bhi
docker images             # images
docker logs -f ekba-dev-postgres-1
docker compose --env-file .env -f infra/docker/docker-compose.yml up -d
docker compose --env-file .env -f infra/docker/docker-compose.yml down
docker build -f infra/docker/backend.Dockerfile -t ekba-backend:local .
```

AWS par Docker Compose **nahi** — wahi chaar images ECS task definition mein containers ban kar chalti hain.

24. Terraform — Beginner Guide

24.1 Basic commands

Command	Matlab
<code>terraform init -backend-config=backend.hcl</code>	Providers download + remote state se jodna
<code>terraform fmt -recursive</code>	Code format
<code>terraform validate</code>	Syntax/logic check (AWS ko touch nahi karta)
<code>terraform plan -out=tfplan</code>	"Kya badlega" — padho!
<code>terraform apply tfplan</code>	Sirf wahi reviewed plan lagao
<code>terraform destroy</code>	Is project mein sirf <code>scripts/destroy.sh</code> ke through

State = Terraform ki diary ki usne kya banaya. **Backend** = state kahan rehti (S3 + DynamoDB lock). **tfvars** = real values (git-ignored). **Module** = reusable block (`modules/`). **Environment** = `envs/<name>` — har ek ki alag state.

24.2 Chaar states

State	Key	Kaun chalata hai	Destroy?
<code>envs/baseline</code>	<code>baseline/terraform.tfstate</code>	Haath se, ek baar	Kabhi nahi
<code>envs/cost-guard</code>	<code>cost-guard/terraform.tfstate</code>	Haath se	Kabhi nahi
<code>envs/dev</code>	<code>dev/terraform.tfstate</code>	Sirf <code>deploy.sh</code> (image + snapshot var deta hai)	Sirf <code>destroy.sh</code> (pehle DB snapshot)
<code>envs/frontend</code>	<code>frontend/terraform.tfstate</code>	Sirf <code>deploy-frontend.sh</code> (ALB + app ID var deta hai)	Kabhi nahi (script nahi)

Har folder mein git-ignored `backend.hcl` (account ID) aur `terraform.tfvars` (real values); commit hota hai sirf `example`.

24.3 "Plan mein destroy/replace dikha — ab kya karu?"

Kabhi blindly apply mat karo. Checklist:

1. **Kaunsa resource?** `terraform show tfplan | grep -E "destroyed|replaced"`
2. **Kyun?** Us resource ke andar `# forces replacement` wali line dhoondho — wahi attribute wajah hai.
3. **Kya ye expected hai?** Neeche table dekho.
4. **Data jayega?** S3, secrets, Cognito, ECR — inka replace **kabhi** accept mat karo.
5. **Unsure ho to rukho.** Plan file delete karo, kuch apply mat karo.

Plan mein	Normal hai?	Kyun
<code>aws_ecs_task_definition.app</code> must be replaced	✅ Haan	Task definition revisions immutable hain; env var/image change = nayi revision
<code>aws_security_group.alb</code> updated in-place	✅	Ingress rule change
<code>aws_amplify_app ...</code> must be replaced	❌ NAHI	Real example is project se: console ne <code>iam_service_role_arn</code> lagaya tha; Terraform use hataana chahta tha → provider app replace karta (GitHub connection khatam). Fix: <code>lifecycle { ignore_changes = [iam_service_role_arn] }</code>
Kuch bhi <code>destroy</code> baseline mein	❌	Baseline protected hai

24.4 State lock atka ho

Error acquiring the state lock → pehle check karo koi aur terraform/deploy chal to nahi raha. Sach mein atka ho tab hi `terraform force-unlock <LOCK_ID>` — ye human decision hai, rule ke hisaab se record karo.

25. CI/CD — GitHub Actions

25.1 `ci.yml` (automatic)

Trigger: push to `main` / `develop`, pull request to `main`. (`aws-deployment` par nahi.)

Job	Steps	Blocking?
backend	ruff lint, mypy, pytest unit/integration/security/evaluation	mypy non-blocking (warning)
frontend	npm ci, typecheck, lint, build, static-export build	Blocking
security	gitleaks secret scan, pip audit, npm audit, checkov (Terraform)	npm audit warning; checkov <code>soft_fail</code>
terraform	fmt -check, validate baseline/dev/cost-guard/frontend	Blocking
docker	backend image build, Trivy scan, non-root check	Blocking

25.2 `deploy.yml` (manual, `workflow_dispatch`)

Type `DEPLOY` → OIDC role → build/push → plan → reject destroys → apply → CodeDeploy → health → verify → rollback on failure.

Abhi tak run nahi hua, aur 3 known problems hain: (1) iska cost check credits ke baad ka spend padhta hai aur deploy role ke paas `ce:GetCostAndUsage` nahi — kabhi block nahi karega; (2) GitHub runner ka IP `DEMO_ALLOWED_CIDR` mein nahi — health check ALB tak nahi pahunchega (CloudFront URL use karna fix hoga); (3) `allowed_cidrs` do jagah se set hota hai. **Backend ke liye `./scripts/deploy.sh` use karo.**

Required GitHub settings (`deploy.yml` ke liye): repo variables `AWS_REGION`, `EXPECTED_AWS_ACCOUNT_ID`, `DEMO_ALLOWED_CIDR`, `S3_BUCKET`, `COGNITO_CLIENT_ID`; secret `AWS_DEPLOY_ROLE_ARN`. Ye set hain ya nahi — repository se confirm nahi ho paya.

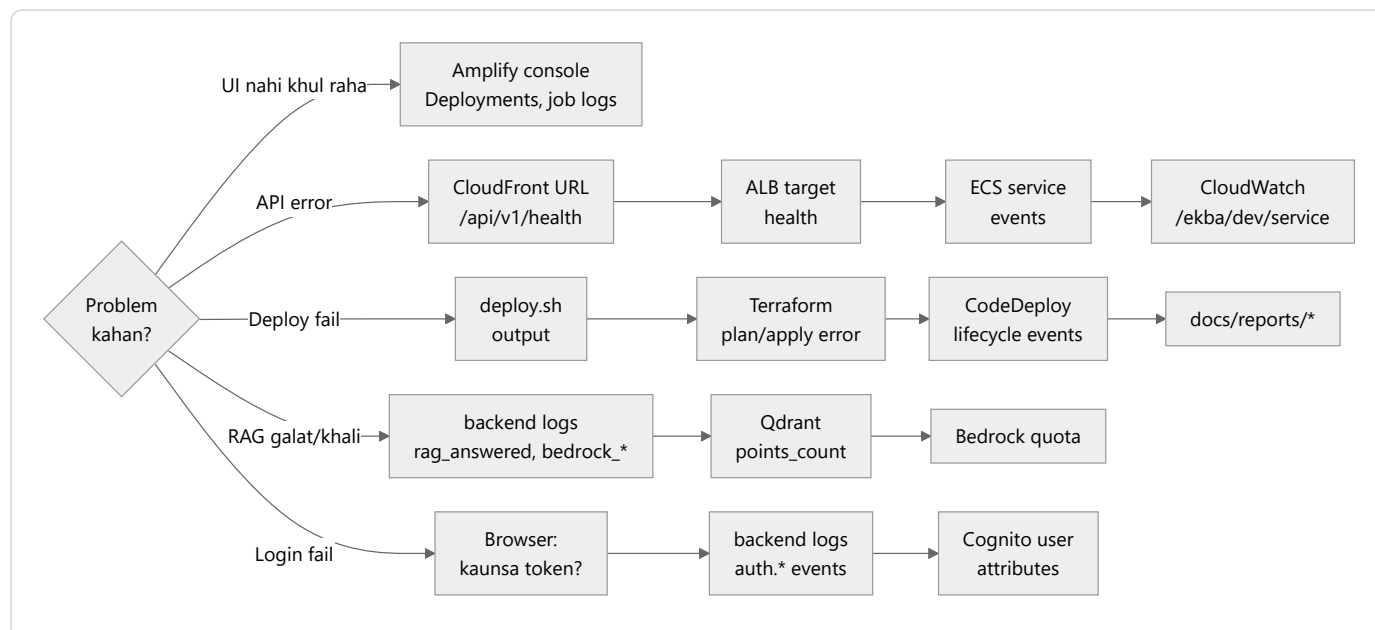
26. Testing

Kya	Command	Kahan se
Backend lint	<code>ruff check app tests seeds</code> · <code>ruff format --check app tests seeds</code>	<code>backend/</code>
Unit	<code>pytest tests/unit -q</code>	mocks, no services
Integration	<code>pytest tests/integration -q</code>	API smoke, RAG pipeline
Security	<code>pytest tests/security -q</code>	tenant isolation, guardrails, platform security, agent isolation
Evaluation	<code>pytest tests/evaluation -q</code>	RAG quality harness
Sab ek saath (deploy gate)	<code>ENVIRONMENT=dev AI_PROVIDER=local DEV_AUTH_ENABLED=true pytest tests/unit tests/integration tests/security tests/evaluation -q</code> → 190 tests	<code>deploy.sh</code>
Frontend	<code>npm run lint</code> · <code>npm run typecheck</code> · <code>npm run build</code>	<code>frontend/</code>
Amplify jaisa build	<code>NEXT_OUTPUT_MODE=export NEXT_PUBLIC_API_URL=https://x.invalid npm run build</code>	<code>frontend/</code>
E2E (Playwright, 15 journeys)	<code>./scripts/test-e2e.sh</code> (tokens khud banata hai) ya <code>npx playwright test</code>	local stack chahiye
Cost-guard Lambda	<code>backend/.venv/Scripts/python.exe -m pytest infra/terraform/modules/cost-guard/lambda/tests -q</code> (27)	repo root
Terraform	<code>terraform fmt -check -recursive infra/terraform</code> · <code>terraform -chdir=... validate</code>	

CODE CHANGE → lint/typecheck → tests → build → commit → deploy (script) → verify (script)

Rule: test ko kabhi weak/delete mat karo build pass karane ke liye — code fix karo.

27. Monitoring & Logs — "Problem Aaya To Sabse Pehle Kahan Dekhu?"



Correlation ID: har request ka `X-Correlation-ID` frontend banata hai; backend har log line, error response aur audit event mein wahi ID rakhta hai. Error ke `correlation_id` se logs mein search karo.

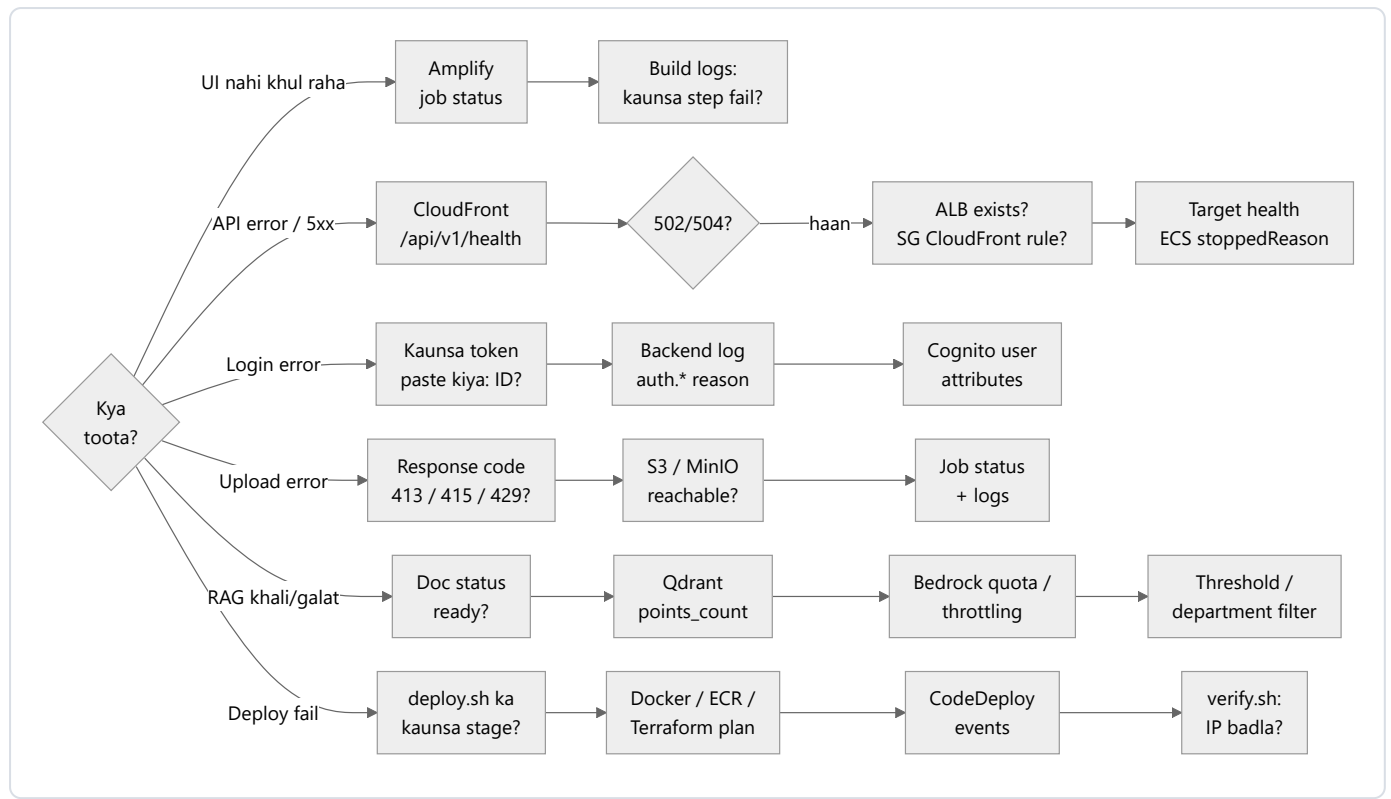
```
MSYS_NO_PATHCONV=1 aws logs filter-log-events --log-group-name /ekba/dev/service \
  --log-stream-name-prefix api --filter-pattern '"<CORRELATION_ID>"' --query 'events[].message' --output
text
```

Aur jagah: admin UI (AI Metrics, Security, Audit pages), `audit_events` table, `./scripts/cost-check.sh`, CloudWatch alarms `ekba-dev-5xx` / `ekba-dev-latency`.

Monitoring ki limitations: custom CloudWatch metrics (tokens, cost, cache hits as metrics) aur LangSmith tracing — **implemented nahi hain**; ye data `request_usage` table aur admin metrics endpoint mein hai.

28. Troubleshooting Playbook

28.1 "Kuch toot gaya" — decision tree



28.2 Problem → Check → Command → Wajah → Fix

Frontend / Amplify

Problem	Check / Command	Possible cause	Fix
Frontend nahi khul raha	<code>aws amplify list-jobs --app-id <APP_ID> --branch-name aws-deployment --query 'jobSummaries[0].[jobId,status]'</code>	Latest build failed	Console → job → BUILD logs
Build fail: <code>NEXT_PUBLIC_API_URL must be an https:// URL, got ''</code>	Build log	App par env var set nahi (console se naya app banane par pehli build)	<code>./scripts/deploy-frontend.sh</code> (Terraform env var set karta hai)
Build fail: Monorepo spec provided without "applications" key	Build log (build "completed successfully" ke baad)	Custom headers galat format mein	<code>modules/frontend/main.tf</code> — headers <code>applications[.appRoot</code> ke neeche (fix commit <code>6a3e2ef</code>)
Page khulta hai par "Could not reach the API"	Browser DevTools → Network	API down / galat <code>NEXT_PUBLIC_API_URL</code> / CORS	Neeche API aur CORS rows

Problem	Check / Command	Possible cause	Fix
Unknown URL par 200 aata hai, 404 nahi	<code>curl -sIL <SITE>/nope</code>	Amplify ka behaviour: 301 → 302 → /404.html → 200	Expected — visitor ko not-found page dikhta hai

API path (CloudFront → ALB → ECS)

Problem	Check / Command	Possible cause	Fix
CloudFront 504	<code>curl -s -o /dev/null -w '%{http_code}' https://<CF>/api/v1/health</code>	ALB SG CloudFront ko allow nahi karta	<code>deploy-frontend.sh</code> (wiring) / <code>deploy.sh</code>
CloudFront 502/504 , ALB gayab	<code>aws elbv2 describe-load-balancers --names ekba-dev-alb</code>	<code>destroy.sh</code> ya armed cost guard ne ALB hataya	<code>./scripts/destroy.sh</code> (cleanup) → <code>deploy.sh</code> → <code>deploy-frontend.sh</code>
ALB targets unhealthy	<code>aws elbv2 describe-target-health --target-group-arn <TG_ARN></code>	Container crash / health path	ECS events + logs; health path <code>/api/v1/health</code> TrustedHost se exempt hai
Target <code>unused</code> mid-deploy	same	Us group par abhi listener nahi	Normal jab tak <code>AllowTestTraffic</code>
ECS task stopped / baar-baar restart	<code>aws ecs describe-tasks --cluster ekba-dev --tasks <TASK_ARN> --query 'tasks[0].[stoppedReason,containers[][name,exitCode,reason]]'</code>	Secret missing, image pull fail, essential container (redis/qdrant) down, ya RDS reachable nahi	CloudWatch <code>/ekba/dev/service</code> stream prefix dekho
<code>verify.sh</code> : FAIL health endpoint, curl exit 28	<code>curl -s https://checkip.amazonaws.com</code> vs <code>grep allowed_cidrs infra/terraform/envs/dev/terraform.tfvars</code>	Tumhara IP badal gaya (ISP)	<code>allowed_cidrs</code> update → <code>deploy.sh</code> (1 in-place SG change)
CORS error browser mein	<code>curl -s -D - -o /dev/null -X OPTIONS https://<CF>/api/v1/me -H "Origin: <SITE>" -H "Access-Control-Request-Method: GET"</code>	Live task definition mein origin nahi	<code>deploy-frontend.sh</code> (frontend.auto.tfvars) → <code>deploy.sh</code>
<code>400 Bad Request</code> har call par	Local: <code>.env</code>	Host <code>TRUSTED_HOSTS</code> mein nahi	Host add karo

Deploy

Problem	Check / Command	Possible cause	Fix
<code>SubscriptionRequiredException</code> (CodeDeploy)	<code>aws freetier get-account-plan-state --region us-east-1</code>	Account Free plan par	Paid plan (IAM wajah nahi)
<code>deploy.sh</code> refuse: uncommitted image inputs	<code>git status -- backend infra/docker/backend.Dockerfile</code>	Image tag commit se match nahi karega	Commit karo
<code>deploy.sh</code> refuse: gross usage ≥ \$18	<code>./scripts/cost-check.sh</code>	Cost guard threshold	Destroy + review spend
Docker build fail	<code>docker version</code>	Docker Desktop band	Start karo

Problem	Check / Command	Possible cause	Fix
ECR push fail	<code>aws ecr get-login-password ...</code>	Login / permission	<code>deploy.sh</code> khud login karta hai; credentials check
Image scan critical	<code>aws ecr describe-image-scan-findings --repository-name ekba-dev-backend --image-id imageTag=<SHA></code>	Base image CVE	Base image update, rebuild
CodeDeploy failed	<code>aws deploy get-deployment-target --deployment-id <ID> --target-id ekba-dev:ekba-dev --query 'deploymentTarget.ecsTarget.lifecycleEvents[][lifecycleEventName,status]'</code> <code>--output text</code>	Green unhealthy	Logs; auto-rollback configured
<code>DeploymentTargetDoesNotExistException</code>	—	ECS target ID format galat	<code>--target-id ekba-dev:ekba-dev</code>
<code>deploy.sh</code> : rollback also failed - <code>ESCALATE TO A HUMAN</code>	<code>scripts/rollback.sh</code> padho	<code>rollback.sh</code> ka CodeDeploy traffic shift implemented nahi hai ("not yet implemented (Phase 5)") — ye sirf <code>verify.sh</code> dobara chalata hai. Asli rollback CodeDeploy ka auto-rollback karta hai	Asli failure (aksar IP) diagnose karo

Auth

Problem	Check	Possible cause	Fix
"Token is missing a tenant assignment."	Token decode (jwt.io jaisa tool mat use karo real token ke liye; locally <code>python -c se</code>)	Access token paste kiya / user par <code>custom:tenant_id</code> nahi	ID token use karo; attribute set karo
"Invalid token."	Backend log <code>auth.invalid_token</code> / <code>auth.bad_audience</code> / <code>auth.bad_issuer</code>	Galat pool/client, dev token AWS par	<code>COGNITO_USER_POOL_ID</code> / client ID match
"Token has expired."	—	1 ghanta (Cognito) / 12 ghante (dev)	Naya token
"Authentication is not configured."	—	<code>COGNITO_USER_POOL_ID</code> khali aur dev auth off	Env set karo
403 admin page	—	Role <code>user</code>	<code>custom:role=admin</code> , naya token

Data / RAG

Problem	Check / Command	Possible cause	Fix
DB connection fail (local)	<code>curl localhost:8000/api/v1/health/ready</code>	<code>DATABASE_URL==...</code> double <code>=</code> , password mismatch	<code>.env</code> fix; mismatch par <code>docker compose ... down -v</code> (data jayega)
Qdrant fail	<code>curl localhost:6333/collections/ekba_chunks</code>	Container down / dimension mismatch	Up karo; dimension badli ho to nayi collection + re-index
Redis fail	<code>docker exec -it ekba-dev-redis-1 redis-cli ping</code>	Container down	Cache bina chalega; rate limit ke liye Redis chahiye
Upload 415	Response	Extension/magic bytes mismatch	Sahi file type
Upload 413	Response	> 50 MB	Chhoti file
Upload 429	Response <code>Retry-After</code>	5 uploads/min	Ruko
Document <code>processing</code> mein atka	<code>/documents/<id>/status</code> , logs <code>ingestion_task_crashed</code>	Bedrock throttling; Transcribe (max 15 min); container restart ne background task maar diya (ingestion same process mein chalti hai)	Logs dekho; دوبارا upload
RAG "I could not find anything..."	Doc <code>ready ? points_count ?</code>	Nothing above threshold (0.35/0.15), department filter, doc processing, Bedrock quota	Doc status + Qdrant check
<code>bedrock_embed_failed ... ThrottlingException</code>	Service Quotas	Quota 0	AWS Support case
Galat tenant ka data dikha	Audit <code>tenant.cross_tenant_*</code> events	Critical incident	Turant roko; token claims check; <code>pytest tests/security -q</code>

Terraform / cost

Problem	Fix
<code>Error acquiring the state lock</code>	Koi aur run chal raha? Nahi to hi <code>terraform force-unlock <LOCK_ID></code> (decision record karo)
Plan mein unexpected destroy/replace	Chapter 24.3 checklist — apply mat karo
Cost badh raha hai	<code>./scripts/cost-check.sh</code> ; bhoola hua stack → <code>./scripts/destroy.sh</code>

Windows shell traps (asli mein hue):

- Git Bash `/ekba/dev/service` ko Windows path bana deta hai → `MSYS_NO_PATHCONV=1 aws logs ...`
- PowerShell 5.1 `-backend-config=backend.hcl` ko dot par tod deta hai → quote karo ya Git Bash use karo
- AWS CLI `--max-items` ke saath `--output text` extra `None` line deta hai (Windows par `\r` bhi) → scripts mein `--max-items` mat lagao, `tr -d '\r'`

29. Security

29.1 Controls — kya kahan

Control	Implementation	File
Authentication	Cognito RS256 JWT (signature, issuer, audience, expiry); dev HS256 sirf dev mein	<code>core/auth.py</code>
Authorization	Roles <code>user</code> / <code>admin</code> ; admin endpoints <code>AdminUser</code> dependency	<code>api/deps.py</code>
Tenant isolation	Token se tenant; har DB query, Qdrant search (mandatory filter), S3 prefix check, cache key mein tenant	<code>repositories.py</code> , <code>vector.py</code> , <code>storage.py</code> , <code>cache.py</code>
Cross-tenant attempt	404/403 response + critical audit event <code>tenant.cross_tenant_document_access</code>	<code>repositories.require_document</code>
Prompt injection	10 pattern families + invisible chars + encoded blobs; block par audit	<code>security/injection.py</code>
Retrieval poisoning	Ingestion par har chunk scan; suspicious chunks search se bahar	<code>ingestion/pipeline.py</code> , <code>vector.py</code>
Output guardrail	System prompt leak, AWS key, private key, JWT, script/iframe/js URI block	<code>rag/guardrails.py</code>
Citation validation	Invented/foreign-tenant citations hatao	<code>rag/guardrails.py</code>
File upload	Allow-list, magic bytes, 50 MB, server-generated key, sanitized name	<code>security/files.py</code>
Rate limits	20 / 10 / 5 per minute per user (Redis) → 429	<code>core/ratelimit.py</code>
Headers (API)	Security headers middleware	<code>core/middleware.py</code>
Headers (site)	HSTS, nosniff, DENY, no-referrer, CSP	Amplify (<code>modules/frontend</code>)
CORS	Explicit allow-list (never <code>*</code>)	<code>main.py</code> + <code>CORS_ALLOWED_ORIGINS</code>
Trusted hosts	<code>TRUSTED_HOSTS</code> (health path exempt for ALB)	<code>core/middleware.py</code>
Network	Tasks sirf ALB se; ALB sirf tumhara IP + CloudFront	<code>modules/network</code>
S3	Public block, SSE, versioning, TLS-only	baseline
Secrets	Secrets Manager, runtime injection, <code>SecretStr</code> , never in state/logs	baseline, <code>config.py</code> , <code>logging.py</code> (redaction)
IAM	Role per component, scoped ARNs	<code>modules/service</code> , baseline
Container	Non-root user (uid 10001)	<code>backend.Dockerfile</code>
Audit	<code>audit_events</code> (reason codes, payload kabhi nahi)	<code>repositories.record_audit</code>

29.2 Honest gaps

Gap	Detail
ALB par HTTPS nahi	Browser traffic CloudFront par HTTPS hai; CloudFront→ALB HTTP

Gap	Detail
API CloudFront se public	Ab kahin se bhi CloudFront ke through reachable — <code>/health</code> ke alawa sab ke liye valid token chahiye
MFA off	Cognito <code>mfa_configuration = "OFF"</code>
CSP <code>'unsafe-inline'</code>	Next.js inline bootstrap scripts ke liye
Next.js 15.1.3 advisory	Amplify build <code>CVE-2025-66478</code> warning dikhata hai (server rendering se related; site static hai) — upgrade alag decision
npm audit	7 vulnerabilities (Amplify log)
WAF	Implemented nahi
Custom domain / ACM	Implemented nahi

29.3 GitHub mein kabhi commit nahi karna (`.gitignore` se)

`.env`, `.env.*` (sirf `.env.example` allowed) · `*.pem *.key *.p12 *.pfx *.crt` · `credentials`, `aws-credentials`, `secrets.json`, `.secrets/` · `*.tfstate*`, `tfplan`, `*.tfplan`, `tfdestroyplan` · `*.tfvars` (sirf `*.tfvars.example`) · `.terraform/` · `backend.hcl` (account ID) · `envs/dev/frontend.auto.tfvars` (`*.tfvars` pattern) · aur **convention se:** `docs/reports/2026*.md` (account ID + live URLs hote hain).

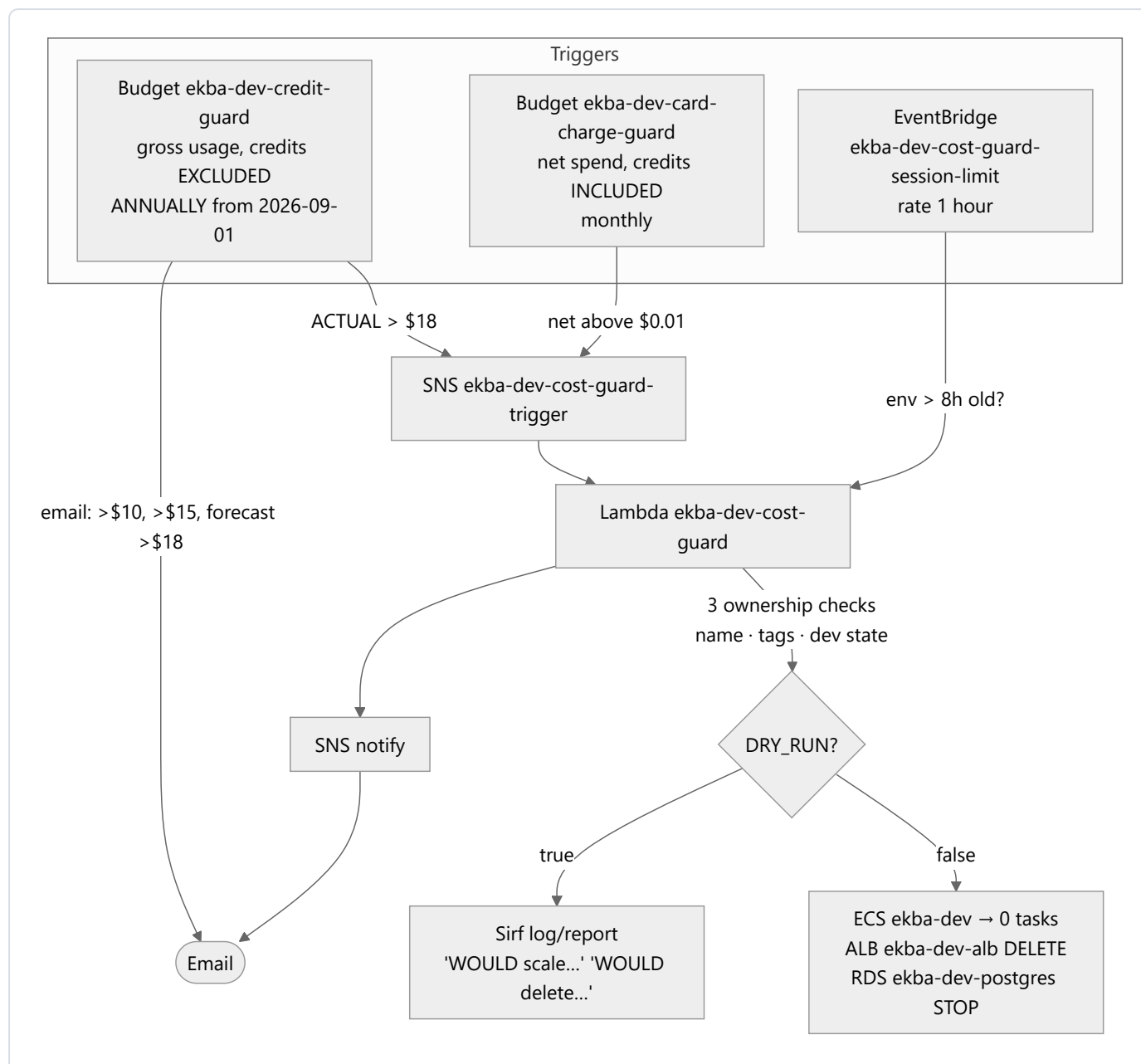
`.env.example` mein sirf placeholders (`<replace-me>`). CI mein gitleaks secret scan chalta hai.

30. Cost Guard — Kharcha Kaise Control Hota Hai

30.1 Pehle ek sach

AWS Budget koi guaranteed hard spending cap nahi hai. Billing data late aata hai (Budgets din mein kuch baar refresh, data ~24 ghante tak late). Isliye shutdown **\$18** par set hai, **\$20 ceiling** se \$2 neeche — asal mein shutdown ~\$18–20 ke beech hota hai.

30.2 Architecture



30.3 Thresholds

Trigger	Threshold	Action
Gross usage (credits + refunds excluded), cumulative	> \$10, > \$15	Email
Same	Forecast > \$18	Email
Same	Actual > \$18	Email + shutdown
Net spend (credits ke baad = card charge), monthly	> \$0.01	Email + shutdown
Hourly check	Environment 8 ghante se purana (ALB/service creation se, redeploy reset nahi karta)	Shutdown (dry-run mein sirf log)

`deploy.sh` bhi **\$18 gross** par deploy refuse karta hai (`COST_GUARD_SHUTDOWN_USD` , `COST_GUARD_START=2026-09-01`).

30.4 Shutdown kya karta hai / kya chhodta hai

Karta: ECS service `ekba-dev` → 0 tasks (delete nahi) · ALB `ekba-dev-alb` + listeners **delete** (ALB pause nahi hota; idle ALB ~\$24/month). **Chhodta:** VPC, subnets, SGs, target groups, cluster, task defs, IAM, logs, alarms (\$0 idle — `destroy.sh` hatata hai) · poora baseline · frontend · state backend. **Kabhi** `terraform destroy` **nahi chalata**.

Safety: har resource ke liye teen checks — naam `ekba-dev` / `ekba-dev-*`, tags `ProjectCode=ekba` + `Environment=dev` + `Lifecycle=ephemeral`, aur ARN `s3://ekba-tfstate-<ACCOUNT_ID>/dev/terraform.tfstate` mein. IAM role bhi sirf yahi do actions allow karta hai (policy simulator se verified).

30.5 DRY_RUN

- **Abhi:** `true` (live Lambda, committed example, local tfvars — teeno).
- Fail-safe: `false` ke alawa koi bhi value = dry run.
- Dry-run mein hourly check sirf log karta hai (email spam nahi); manual/budget trigger email bhejte hain.

Arm karna (jab tum decide karo):

```
# infra/terraform/envs/cost-guard/terraform.tfvars, line 22: dry_run = false
cd infra/terraform/envs/cost-guard
terraform plan -out=tfplan      # expect: 0 to add, 1 to change, 0 to destroy (Lambda env)
terraform apply tfplan
cd ../../.. && ./scripts/cost-check.sh    # "kill switch ... is ARMED"
```

Git push se ye apply **nahi** hota — `terraform.tfvars` git-ignored hai aur koi workflow cost-guard apply nahi karta. Arm karte hi agli hourly run 8 ghante se purana stack band kar degi. Demo chal raha ho to pehle `destroy.sh` + `deploy.sh` (fresh 8 ghante).

30.6 Manually verify

```
./scripts/cost-check.sh # gross / credits / net + ARMED ya DRY RUN
aws lambda invoke --function-name ekba-dev-cost-guard \
  --cli-binary-format raw-in-base64-out --payload '{"trigger":"manual"}' out.json && cat out.json
MSYS_NO_PATHCONV=1 aws logs tail /aws/lambda/ekba-dev-cost-guard --since 1h
```

Dry-run result (2026-09-12): `WOULD scale ECS service ekba-dev/ekba-dev from 1 to 0 tasks` · `WOULD delete load balancer ekba-dev-alb (and its listeners)` · `WOULD stop RDS instance ekba-dev-postgres (status available; data and storage kept)` · refused 0 · errors 0.

Shutdown ke baad: `./scripts/destroy.sh` → `./scripts/deploy.sh` → `./scripts/deploy-frontend.sh` (sirf `deploy.sh` service ko 0 par chhod dega kyunki Terraform `desired_count` ignore karta hai).

30.7 Kharcha — numbers

Cheez	Cost
Backend stack chalte hue	~\$0.11/hour (Fargate ~\$0.054, ALB ~\$0.023 + LCU, 3 public IPv4 ~\$0.015, RDS ~\$0.019) → 4 ghante ~\$0.44, mahina bhool gaye to ~\$80
RDS akela, 24/7	~\$14/month (\$11.68 instance + \$2.30 storage) — isiliye session ke baad <code>destroy.sh</code>
Database snapshots (destroy ke beech)	~\$0.095 per GB per month — yahan kuch cents
Baseline idle	~\$1.70/month (4 secrets × \$0.40)
Cost guard	~\$0 (free tiers)
Amplify	~\$0.04/build, idle ~\$0
CloudFront	demo traffic par ~\$0
Cost Explorer API	\$0.01 per call (<code>deploy.sh</code> , <code>cost-check.sh</code>)

Baseline budget `ekba-dev-monthly` (\$20, 50/80/100%) credits **include** karta hai → credits khatam hone tak \$0 dikhata hai. Asli protection upar wale gross/net budgets hain.

31. Manual Data Inspection (read-only)

Sab commands **read-only** hain. Secrets ki value kabhi print mat karo. AWS par RDS private subnets mein hai (SG sirf task SG se) aur Qdrant/Redis task ke andar hain (task SG sirf ALB se), ECS Exec off — wahan API aur logs use karo.

S3

```
aws s3 ls s3://<BUCKET>/ --recursive --human-readable | head          # AWS
# Local MinIO console: http://localhost:9001 (credentials .env mein)
```

PostgreSQL (local)

```
docker exec -it ekba-dev-postgres-1 psql -U ekba -d ekba
```

```
\dt
select tenant_id, status, count(*) from documents group by 1,2;
select document_id, status, progress, error_code from ingestion_jobs order by created_at desc limit 10;
select operation, model_used, input_tokens, output_tokens, estimated_cost, cache_hit
  from request_usage order by created_at desc limit 10;
select event_type, severity, reason, created_at from audit_events order by created_at desc limit 20;
```

Qdrant (local)

```
curl -s http://localhost:6333/collections/ekba_chunks          # points_count, config
curl -s -X POST http://localhost:6333/collections/ekba_chunks/points/scroll \
  -H 'Content-Type: application/json' \
  -d '{"limit":3,"with_payload":true,"with_vector":false,
      "filter":{"must":[{"key":"tenant_id","match":{"value":"seed-tenant-northwind"}}]}}'
# UI: http://localhost:6333/dashboard
```

Redis (local)

```
docker exec -it ekba-dev-redis-1 redis-cli --scan --pattern 'ekba:*' | head
docker exec -it ekba-dev-redis-1 redis-cli ttl <KEY>
```

Cognito (AWS)

```
aws cognito-idp list-users --user-pool-id "$POOL" --query 'Users[][Username,UserStatus]' --output table
aws cognito-idp admin-get-user --user-pool-id "$POOL" --username <EMAIL> --query 'UserAttributes'
```

ECS / CloudWatch (AWS)

```
aws ecs describe-services --cluster ekba-dev --services ekba-dev --query 'services[0].events[:5].
[createdAt,message]' --output text
MSYS_NO_PATHCONV=1 aws logs tail /ekba/dev/service --since 30m
```

AWS par app data: admin token se `GET /api/v1/admin/metrics`, `GET /api/v1/documents`, `GET /api/v1/admin/audit`.

32. Real End-to-End Example — "ABC Insurance"

Ye story **currently supported** steps use karti hai. AWS par step 10 aur 20 abhi Bedrock quota (0) ki wajah se fail honge; local mode mein poora flow local AI stub ke saath chalta hai.

#	Kya hota hai	Kaun/kahan
1	Tenant ID choose: <code>tenant-abc-insurance</code>	Tum (koi API nahi)
2	Admin banao: <code>admin-create-user</code> with <code>custom:tenant_id</code> , <code>custom:role=admin</code>	Cognito CLI
3	Password set, admin ko out-of-band do	<code>admin-set-user-password</code>
4	Admin ID token leta hai, frontend mein paste	<code>initiate-auth</code> → sign-in box
5	Documents page → "HR Maternity Policy.pdf" upload (dept <code>hr</code>)	<code>POST /api/v1/documents</code>
6	File S3: <code>tenant-abc-insurance/<uuid>.pdf</code> (SSE)	<code>storage.put_object</code>
7	Rows: <code>documents</code> pending, <code>ingestion_jobs</code> queued, audit <code>document.uploaded</code>	Postgres (AWS: RDS — ye rows task restart ke baad bhi rehti hain)
8	Background: text extract (pypdf, page numbers ke saath)	<code>extractors.extract_pdf</code>
9	Chunks (512 tokens, 64 overlap) + injection scan	<code>chunker</code> , <code>scan_content</code>
10	Embeddings (Titan v2, batch 25)	Bedrock
11	Qdrant upsert — har point par <code>tenant_id=tenant-abc-insurance</code>	<code>vector.upsert_chunks</code>
12	Document <code>ready</code> , <code>chunk_count</code> set, cache invalidate	Postgres (RDS), Redis
13	User banao (<code>custom:role=user</code>)	Cognito CLI
14	User login (ID token paste)	Frontend
15	Sawaal: "Meri maternity leave policy kya hai?"	Chat page → <code>POST /api/v1/chat</code>
16	JWT verify → <code>tenant_id=tenant-abc-insurance</code> , role user	<code>core/auth.py</code>
17	Validation + injection scan + embed + cache lookup (ABC namespace)	pipeline stages 2–4
18	Qdrant search sirf ABC ke chunks → threshold → rerank	stages 5–10
19	Context envelope → Nova Lite	stages 8–9
20	Citation validation + output guardrail + confidence	stages 11–12
21	Usage + conversation save; answer (language of question) + <code>[S1]</code> → "HR Maternity Policy.pdf, page N"	<code>chat.py</code>
22	Frontend answer, citations, confidence, model, tokens, cost dikhata hai	<code>app/chat/page.tsx</code>

ADMIN kar sakta: upload, tenant ka koi bhi document delete, chat, workflows, AI metrics, audit, security events.

USER kar sakta: upload, apna document delete, chat, workflows, feedback. Admin pages nahi. **Koi nahi kar sakta**

(app mein): users/tenants manage karna, doosre tenant ka data dekhna.

33. Complete Command Cheat Sheet

Sab commands **repo root** (D:\infy__masters) se, Git Bash mein — jab tak `cd` na likha ho. `<...>` wali values apni daalo; secrets kabhi command line par mat likho.

Local development

Kaam	Command
Ek-command setup	<code>./scripts/setup-local.sh</code> (PowerShell: <code>.\scripts\setup-local.ps1</code>)
Containers start	<code>docker compose --env-file .env -f infra/docker/docker-compose.yml up -d postgres qdrant redis minio minio-init</code>
Containers status	<code>docker compose --env-file .env -f infra/docker/docker-compose.yml ps</code>
Containers stop (data rahe)	<code>docker compose --env-file .env -f infra/docker/docker-compose.yml stop</code>
Containers + data delete	<code>... down -v</code>
venv activate	<code>cd backend && source .venv/Scripts/activate</code>
Migrations	<code>cd backend && alembic upgrade head</code>
Seed data	<code>cd backend && python -m seeds.seed</code>
Backend run	<code>cd backend && uvicorn app.main:app --reload --port 8000</code>
Frontend run	<code>cd frontend && npm run dev</code>
Dev token (user / admin)	<code>cd backend && python -m seeds.dev_token / ... --role admin --user seed-admin-a</code>
Health	<code>curl localhost:8000/api/v1/health</code> · <code>curl localhost:8000/api/v1/health/ready</code>

Testing

Kaam	Command
Backend lint + format	<code>cd backend && ruff check . && ruff format --check .</code>
Backend sab tests	<code>cd backend && ENVIRONMENT=dev AI_PROVIDER=local DEV_AUTH_ENABLED=true pytest tests/unit tests/integration tests/security tests/evaluation -q</code>
Sirf security	<code>cd backend && pytest tests/security -q</code>
Frontend	<code>cd frontend && npm run lint && npm run typecheck && npm run build</code> (dev server band karke)
Cost-guard Lambda	<code>backend/.venv/Scripts/python.exe -m pytest infra/terraform/modules/cost-guard/lambda/tests -q</code>

AWS lifecycle

Kaam	Command
Kharcha + kya chal raha hai	<code>./scripts/cost-check.sh</code>
Backend deploy	<code>./scripts/deploy.sh</code>
Verify	<code>./scripts/verify.sh</code>
Seed (AWS)	<code>./scripts/seed.sh</code>
E2E	<code>./scripts/test-e2e.sh</code>
Backend rollback	<code>CodeDeploy auto-rollback · ya git revert <BAD_SHA> + ./scripts/deploy.sh</code> (Chapter 22.5)
<code>rollback.sh</code>	Sirf verify + report — traffic shift implemented nahi
Frontend deploy / verify	<code>git push origin aws-deployment · ./scripts/deploy-frontend.sh</code>
Frontend rollback	<code>./scripts/rollback-frontend.sh</code> (ya <code>--to <COMMIT_SHA></code>)
Destroy backend	<code>./scripts/destroy.sh</code> → <code>DESTROY ekba-dev</code>
Identity check	<code>aws sts get-caller-identity --query Account --output text</code>
Mera IP	<code>curl -s https://checkip.amazonaws.com</code>

Diagnose (read-only)

Kaam	Command
ECS service	<code>aws ecs describe-services --cluster ekba-dev --services ekba-dev --query 'services[0].[status,runningCount,desiredCount]'</code>
ECS events	<code>aws ecs describe-services --cluster ekba-dev --services ekba-dev --query 'services[0].events[:5].[createdAt,message]' --output text</code>
Backend logs	<code>MSYS_NO_PATHCONV=1 aws logs tail /ekba/dev/service --since 30m</code>
Ek request trace	<code>MSYS_NO_PATHCONV=1 aws logs tail /ekba/dev/service --since 1h --filter-pattern '<CORRELATION_ID>'"</code>
Target health	<code>aws elbv2 describe-target-health --target-group-arn <TG_ARN></code>
CodeDeploy latest	<code>aws deploy list-deployments --application-name ekba-dev --deployment-group-name ekba-dev-dg --query 'deployments[0]' --output text</code>
CodeDeploy status	<code>aws deploy get-deployment --deployment-id <DEPLOYMENT_ID> --query 'deploymentInfo.[status,errorInformation]'</code>
Amplify jobs	<code>aws amplify list-jobs --app-id <APP_ID> --branch-name aws-deployment --query 'jobSummaries[:3].[jobId,status]'</code>
CloudFront → API	<code>curl -s https://<CLOUDFRONT_DOMAIN>/api/v1/health</code>
Cost-guard logs	<code>MSYS_NO_PATHCONV=1 aws logs tail /aws/lambda/ekba-dev-cost-guard --since 1h</code>
RDS status	<code>aws rds describe-db-instances --db-instance-identifier ekba-dev-postgres --query 'DBInstances[0].[DBInstanceStatus,PubliclyAccessible,StorageEncrypted]' --output text</code>

Kaam	Command
RDS snapshots	<code>aws rds describe-db-snapshots --db-instance-identifier ekba-dev-postgres --snapshot-type manual --query 'DBSnapshots[].[DBSnapshotIdentifier,Status]' --output text</code>
Database reachable? (task se)	<code>curl -s https://<CLOUDFRONT_DOMAIN>/api/v1/health/ready — postgres component dekho</code>
Terraform outputs	<code>cd infra/terraform/envs/<env> && terraform output</code>

Git

Kaam	Command
Status	<code>git status</code>
Kya badla	<code>git diff / git diff --cached --check</code>
Commit (files naam se)	<code>git add <files> && git commit -m "<message>"</code>
Frontend release	<code>git push origin aws-deployment</code>
Branch compare	<code>git rev-list --count origin/main..aws-deployment</code>

34. Interview Preparation

34.1 Two-minute version

"Maine ek **multi-tenant enterprise RAG platform** banaya hai. Companies apne documents — PDF, Word, Excel, images, audio — upload karti hain, aur employees natural language mein sawaal poochte hain; jawab **citations aur confidence score** ke saath aata hai.

Backend **FastAPI** hai. Upload par file validate hoti hai (extension + magic bytes), S3 mein encrypted jaati hai, text extract hota hai, 512-token chunks bante hain, **Titan v2** se 1024-dim embeddings **Qdrant** mein jaati hain. Query par ek **14-stage pipeline** chalti hai: injection scan, semantic cache, tenant-filtered vector search, relevance threshold, BM25 rerank, **Amazon Nova** se answer, phir citation validation aur output guardrail.

Sabse important cheez **tenant isolation** hai — `tenant_id` sirf verified **Cognito JWT** se aata hai aur har DB query, har Qdrant search, har cache key aur har S3 prefix mein lagta hai.

AWS par **ECS Fargate** chalta hai, **CodeDeploy blue/green** ke saath auto-rollback; frontend **Amplify** par static export, API **CloudFront** ke through HTTPS. Sab **Terraform** mein — chaar states. Budget \$20 tha, isliye ephemeral design: NAT aur ElastiCache nahi; Qdrant aur Redis containers hain, aur PostgreSQL ek chhota **RDS** (`db.t4g.micro`) jo stack ke saath hi banta-mitta hai — uska data snapshot ke through agle session mein wapas aa jaata hai. Ek **cost-guard Lambda** hai jo budget cross hone ya 8 ghante baad stack band kar sakta hai."

34.2 Five-minute version — structure

- Problem** (30 s): company knowledge scattered; generic chatbots hallucinate aur tenants ka data mix kar sakte hain.
- Architecture** (60 s): Browser → Amplify (static Next.js) · API → CloudFront → ALB → ECS task (api + qdrant + redis) → RDS PostgreSQL (private subnets), Bedrock, S3, Cognito, Transcribe.
- Ingestion** (45 s): validation, S3 key `<tenant>/<uuid>`, extractors, chunker, per-chunk injection scan (retrieval poisoning), batch embeddings, Qdrant with `tenant_id` payload index.
- RAG + guardrails** (75 s): 14 stages; context envelope (documents = data, not instructions); citations `[Sn]` validate; confidence formula; semantic cache per tenant ($\cosine \geq 0.95$); daily cost ceiling per user.
- Security** (45 s): JWT RS256 via JWKS; roles; rate limits; audit events; security headers; non-root container; secrets in Secrets Manager.
- DevOps** (45 s): Terraform 4 states; blue/green with 5xx alarm rollback; CI (lint, tests, gitleaks, Trivy, checkov, terraform validate).
- Honest limits** (30 s): Bedrock quota 0 is live par; relational data ab RDS mein bachta hai par Qdrant vectors abhi bhi task ke saath jaate hain; no self-service signup; hosted-UI login nahi; CI `main` par.

34.3 Likely questions — short answers

Question	Answer
RAG kya hai?	Model ko sawaal ke saath relevant document chunks dena taaki jawab un par based ho, training data par nahi
Qdrant kyun?	Payload filtering (tenant) + open source + container mein chal jaata hai (\$0)
Tenant isolation kaise?	Tenant token se; har layer (DB, Qdrant <code>must</code> filter, Redis key, S3 prefix) mein; cross-tenant attempt par critical audit; <code>tests/security</code>

Question	Answer
Hallucination kaise kam?	Relevance threshold, "not found" refusal, citation validation, confidence score, strict prompt
Prompt injection?	Input scan (10 families), chunk scan at ingestion, envelope, output guardrail
Semantic cache?	Question embedding ka cosine ≥ 0.95 match same tenant mein; cache hit par bhi auth + guardrail
Blue/green?	Naya task set test listener (:8080) par; healthy hone par traffic shift; 5 minute rollback window; 5xx alarm par auto-rollback
Database kaunsa?	AWS par RDS PostgreSQL (<code>db.t4g.micro</code> , private, encrypted, password Secrets Manager se write-only argument ke through). Pehle sidecar container tha — data har task restart par chala jaata tha
Database sirf ek chhota instance kyun?	\$20 ceiling. Single-AZ <code>db.t4g.micro</code> ~\$14/month 24/7, isliye wo ephemeral stack ke saath hi rehta hai aur <code>destroy.sh</code> snapshot le leta hai. Production mein Multi-AZ + read replica + ElastiCache
Scale kaise?	Stateless API ke multiple tasks; managed DB/Qdrant Cloud; ingestion ko SQS worker mein nikalna
Biggest challenge?	CodeDeploy Free plan block; IP change par health fail; Amplify monorepo headers — har ek ka root cause dhoondha
Kya improve karoge?	Hosted UI login + refresh, tenant/user admin API, SQS ingestion worker, WAF + custom domain, <code>main</code> -based CI/CD

Interview mein **wahi bolo jo implemented hai**. "Async workers", "LangSmith tracing", "Cognito hosted UI login" — ye abhi **implemented nahi** hain; poochha jaye to "planned" bolo.

35. NEVER DO THIS

In mein se koi bhi galti data leak, kharcha ya toota hua deploy de sakti hai.

1. **Secrets commit mat karo** — `.env`, `*.tfvars`, `*.tfstate`, `tfplan`, `backend.hcl`, keys, tokens.
2. **git add . mat karo** — `docs/reports/2026*.md` (account ID + live URLs) andar aa jaati hain. Files naam se add karo.
3. **terraform destroy seedha mat chalao** — sirf `./scripts/destroy.sh`.
4. **Baseline / cost-guard / frontend / state backend destroy mat karo** — secrets, ECR images, Cognito users, budget, URLs sab jayenge.
5. **Destroy/replace wala plan bina samjhe apply mat karo** — khaaskar Amplify app replace (naya URL, GitHub reconnect).
6. **dry_run = false bina soche mat karo** — agli hourly run 8 ghante purana stack band karegi.
7. **AWS Budget ko hard cap mat samjho.**
8. **Frontend se tenant_id bhejne wala endpoint mat banao** — tenant sirf JWT se.
9. **User ka custom:tenant_id galti se mat badlo** — wo user doosri company ka data dekhne lagega.
10. **Guardrail stage skip mat karo** "speed ke liye", aur test delete karke build green mat karo.
11. **NEXT_PUBLIC_* mein secret mat daalo** — browser mein sab public hai.
12. **Real JWT online decoder mein paste mat karo.**
13. **ECS ko haath se scale / console se edit mat karo** — Terraform drift; ALB ka kharcha chalta rehta hai.
14. **Dev server chalte hue npm run build mat chalao.**
15. **docker compose down -v bina soche mat chalao** — local DB, Qdrant, MinIO data delete.
16. **Embedding model badal kar same Qdrant collection use mat karo** — nayi collection + re-index.
17. **Stack chalu chhod kar mat jao** — session ke end mein `./scripts/destroy.sh`.
18. **Kisi aur project ke AWS resources ko touch mat karo** — ownership = state + `ekba-*` naam + `ProjectCode=ekba` tag.

36. Glossary

Term	Simple matlab (is project mein)
RAG	Retrieval-Augmented Generation — pehle documents se relevant hisse dhoondo, phir model unke basis par jawab de
LLM	Large Language Model — yahan Amazon Nova Lite / Micro
Embedding	Text ka number-vector (1024 numbers, Titan v2) — similar meaning = paas-paas vectors
Vector DB	Vectors store + similarity search — yahan Qdrant
Chunk	Document ka chhota tukda (~512 tokens) jo embed hota hai
Token	Text ki unit (~4 characters); model ka kharcha tokens par
Cosine similarity	Do vectors kitne same direction mein — 1 = same meaning
Rerank	Vector results ko dobara order karna (yahan BM25 lexical mix)
Semantic cache	Milte-julte sawaal ka pichla jawab reuse (same tenant)
Tenant	Ek company; uska data baaki se alag
JWT	Signed token jisme user ki claims (tenant, role) hoti hain
ID token vs access token	Cognito ke do token; backend ko ID token chahiye (usme <code>aud</code> + custom claims)
JWKS	Cognito ki public keys jinse token signature verify hota hai
Cognito	AWS user directory + login
Guardrail	Safety check stage (input, retrieval, output)
Prompt injection	Text jo model ko instructions badalne ke liye bahkaaye
Citation <code>[Sn]</code>	Jawab ka woh hissa kis source chunk se aaya
ECS / Fargate	AWS container service / bina server manage kiye containers
Task definition	Container recipe (image, CPU, memory, env, secrets)
ALB	Application Load Balancer — traffic ko healthy tasks par bhejta
Target group	ALB ke peeche tasks ki list (blue / green)
CodeDeploy blue/green	Naya version saath mein chalao, check karo, traffic shift karo
CloudFront	AWS CDN — yahan API ko HTTPS dene ke liye
Amplify	Static frontend hosting + GitHub se auto-build
Static export	Next.js ko plain HTML/JS files mein build karna (<code>NEXT_OUTPUT_MODE=export</code>)
CORS	Browser rule — kaunsi website API call kar sakti hai
Terraform state	Terraform ki yaaddasht — kaunsa resource usne banaya
Plan / apply	Kya badlega dikhana / sach mein badalna
Drift	Asli AWS aur Terraform state mein farak (haath se change karne se)

Term	Simple matlab (is project mein)
Ephemeral	Temporary — destroy + recreate by design
DRY_RUN	Cost guard sirf batata hai, kuch band nahi karta
Correlation ID	Ek request ka unique ID, har log line mein
Alembic	Database schema migrations tool
Idempotent	Dobara chalane par same result, koi nuksaan nahi

37. Final Quick Reference

Cheez	Value
Frontend (live)	<code>https://aws-deployment.d39pgpamq0p0n8.amplifyapp.com</code>
API (live, jab backend deployed ho)	<code>https://d2jw2wchz5oekh.cloudfront.net/api/v1</code>
Local	Frontend <code>http://localhost:3000</code> · API <code>http://localhost:8000</code> · Docs <code>/docs</code> · Qdrant <code>:6333/dashboard</code> · MinIO <code>:9001</code>
Region / states	<code>us-west-2</code> · baseline, cost-guard, dev, frontend
Release branch	<code>aws-deployment</code> (Amplify) · CI <code>main</code> par
Token	Cognito ID token (AWS) · <code>python -m seeds.dev_token</code> (local)
Cost guard	<code>DRY_RUN=true</code> · \$18 gross shutdown · card > \$0.01 · 8 ghante

Roz ka flow

```

./scripts/cost-check.sh      # 1. kya chal raha hai, kitna kharcha
./scripts/deploy.sh          # 2. backend up (commit ke baad)
./scripts/deploy-frontend.sh # 3. frontend wiring verify (12 checks)
# 4. demo
./scripts/destroy.sh         # 5. session khatam — DESTROY ekba-dev

```

Problem aaye to — pehle ye 5

1. `./scripts/cost-check.sh` — stack chal bhi raha hai?
2. `curl -s https://checkip.amazonaws.com` vs `allowed_cidrs` — IP badla?
3. `MSYS_NO_PATHCONV=1 aws logs tail /ekba/dev/service --since 30m`
4. ECS service events (`describe-services ... events[:5]`)
5. Amplify job logs / CodeDeploy lifecycle events

Files jo sabse zyada kaam aayengi

`backend/app/services/rag/pipeline.py` (RAG) · `backend/app/core/auth.py` (login) · `backend/app/core/config.py` (settings) · `frontend/src/lib/api.ts` (API client) · `infra/terraform/envs/dev/terraform.tfvars` (IP, sizing) · `infra/terraform/envs/cost-guard/terraform.tfvars` (DRY_RUN) · `RUNBOOK.md` · `scripts/`

Mujhe ye karna hai — kaunsa chapter?

Kaam	Chapter	Kaam	Chapter
Project local start	18	Backend change / deploy	20.1 / 19.2
Users banana / use karna	7, 6	Logs dekhna	27
Admin vs user	8	API troubleshoot	28.2
Document add karna	12, 32	Cognito / login troubleshoot	6.6, 28.2
Data kahan jaata hai	17	Qdrant troubleshoot	15, 28.2
Sawaal poochna	13.1, 32	Frontend / backend stop	18.5, 22, 22.5

Kaam	Chapter	Kaam	Chapter
RAG flow samajhna	13	AWS band karna	22
Data inspect karna	31	Redeploy	22.4
Frontend change	20.2	Rollback	22.5
Frontend deploy	19.1, 22.2	Safe destroy	22.3, 24.3
AWS kharcha	30	Interview	34

Yaad rakho: Local = \$0, sab kuch pehle local mein. AWS = demo ke liye, aur **demo ke baad** `destroy.sh` .